

Kompilatory

May 31, 2026

1 Štruktúra a použitie kompilátorov, nástroje pre tvorbu kompilátorov

1.1 Základná definícia kompilátora

Kompilátor je program, ktorý prekladá zdrojový program zo vstupného jazyka do cieľového programu vo výstupnom jazyku. Nech L_{in} je vstupný jazyk generovaný gramatikou G_{in} a L_{out} je výstupný jazyk generovaný gramatikou G_{out} . Kompilátor možno chápať ako zobrazenie

$$L_{in} \rightarrow L_{out},$$

pričom pre každý korektný vstup $w_{in} \in L_{in}$ existuje výstup $w_{out} \in L_{out}$. Ak $w_{in} \notin L_{in}$, preklad neexistuje a kompilátor má oznámiť chybu.

1.2 Spracovanie programu

Kompilátor je často súčasťou väčšieho prekladového reťazca:

zdrojový program $\rightarrow$ preprocesor $\rightarrow$ kompilátor $\rightarrow$ assembler $\rightarrow$ linker/loader $\rightarrow$ spustiteľný program.

Preprocesor vykonáva napríklad spracovanie makier a vkladanie súborov. Assembler prekladá assembler do relokovateľného strojového kódu a linker spája objektové súbory a knižnice do výsledného programu.

1.3 Fázy kompilácie

Typický kompilátor obsahuje tieto fázy:

1. lexikálna analýza,
2. syntaktická analýza,
3. sémantická analýza,
4. generovanie medzikódu,

5. optimalizácia medzikódu,
6. generovanie cieľového kódu.

Naprieč fázami sa používa tabuľka symbolov a mechanizmus spracovania chýb.

1.3.1 Lexikálna analýza

Lexikálna analýza číta znaky zdrojového programu a zoskupuje ich do tokenov. Token je kategória lexém so spoločným významom, napríklad identifikátor, číslo, kľúčové slovo alebo operátor.

Príklad:

`dráha := počiatočnosť + čas * 60`

môže byť prevedený na postupnosť tokenov

`id := id + id * number.`

Tokeny sa typicky definujú regulárnymi výrazmi. Napríklad identifikátor:

$$\begin{aligned} id &\rightarrow letter(letter \mid digit)^*, \\ letter &\rightarrow A \mid \dots \mid Z \mid a \mid \dots \mid z, \\ digit &\rightarrow 0 \mid \dots \mid 9. \end{aligned}$$

Rozpoznávanie tokenov teda zodpovedá rozpoznávaniu regulárneho jazyka, ktoré možno realizovať konečným automatom.

1.3.2 Syntaktická analýza

Syntaktická analýza spracúva postupnosť tokenov a overuje, či má štruktúru popísanú gramatikou jazyka. Výstupom je syntaktický strom alebo abstraktný syntaktický strom.

Syntax väčšiny programovacích jazykov sa opisuje bezkontextovou gramatikou. Napríklad:

$$\begin{aligned} assignment &\rightarrow id := expr, \\ expr &\rightarrow expr + term \mid expr - term \mid term, \\ term &\rightarrow term * factor \mid term / factor \mid factor, \\ factor &\rightarrow digit \mid id \mid (expr). \end{aligned}$$

Pre vstup

`id1 := id2 + id3 * 60`

má syntaktický strom vyjadrovať, že násobenie má vyššiu prioritu ako sčítanie:

`id1 := id2 + (id3 * 60).`

Všeobecné algoritmy syntaktickej analýzy bezkontextových jazykov, napríklad CYK alebo Earleyho algoritmus, majú zložitosť $O(n^3)$. Pre praktické programovacie jazyky sa používajú špeciálne triedy gramatík, napríklad LL a LR, ktoré umožňujú lineárnu analýzu.

1.3.3 Sémantická analýza

Sémantická analýza kontroluje vlastnosti programu, ktoré nie je vhodné alebo možné vyjadriť bezkontextovou gramatikou. Ide napríklad o:

- kontrolu deklarácií identifikátorov,
- kontrolu typov operandov,
- kontrolu počtu a typov argumentov funkcií,
- kontrolu návratových hodnôt,
- povolené implicitné typové konverzie.

Program môže byť syntakticky správny, ale sémanticky chybný. Napríklad:

$$x := y + 1$$

je syntakticky správne priradenie, ale ak premenná y nebola deklarovaná, ide o sémantickú chybu.

1.3.4 Tabuľka symbolov

Tabuľka symbolov je dátová štruktúra, v ktorej kompilátor uchováva informácie o identifikátoroch. Pre premenné môže obsahovať meno, typ, rozsah platnosti a umiestnenie v pamäti. Pre funkcie môže obsahovať počet a typy parametrov, spôsob odovzdávania parametrov a návratový typ.

Dôležité požiadavky na tabuľku symbolov sú rýchle vkladanie, vyhľadávanie a modifikácia záznamov.

1.3.5 Spracovanie chýb

Chyby môžu vzniknúť vo všetkých fázach kompilácie:

- lexikálna chyba: neznámy znak alebo postupnosť znakov,
- syntaktická chyba: postupnosť tokenov nespĺňa gramatiku,
- sémantická chyba: syntakticky správna konštrukcia nemá správny význam.

Každá fáza by mala byť schopná chybu oznámiť a podľa možnosti sa zotaviť, aby bolo možné nájsť aj ďalšie chyby.

1.3.6 Generovanie medzikódu

Medzikód je prechodná reprezentácia programu medzi front-endom a back-endom. Mal by byť ľahko generovateľný zo syntaktického stromu, relatívne nezávislý od cieľového stroja a vhodný na optimalizácie.

Často sa používa trojadresový kód:

$$x := y \text{ op } z.$$

Inštrukcia môže byť reprezentovaná štvoricou:

$$(op, arg_1, arg_2, result).$$

Príklad:

$$\begin{aligned} temp1 &:= id3 * 60.0, \\ id1 &:= id2 + temp1. \end{aligned}$$

1.3.7 Optimalizácia medzikódu

Optimalizácia medzikódu transformuje medzikód na efektívnejšiu podobu. Optimalizovať možno rýchlosť, veľkosť kódu alebo využitie pamäte.

Príklady optimalizácií:

- eliminácia mŕtveho kódu,
- eliminácia spoločných podvýrazov,
- konštantové skladanie,
- zjednodušovanie výrazov,
- optimalizácie cyklov.

1.3.8 Generovanie cieľového kódu

Generátor cieľového kódu prekladá medzikód do cieľového jazyka, napríklad assembleru alebo strojového kódu. Rieši výber inštrukcií, prácu s registrami, umiestnenie premenných v pamäti a generovanie skokov a volaní funkcií.

Základnou technikou je *template matching*, kde sa inštrukcie medzikódu nahrádzajú vzormi cieľových inštrukcií.

1.4 Front-end a back-end

Kompilátor sa často delí na front-end a back-end.

Front-end Front-end je časť kompilátora závislá od zdrojového jazyka. Zahŕňa najmä lexikálnu, syntaktickú a sémantickú analýzu a generovanie medzikódu.

Back-end Back-end je časť kompilátora závislá od cieľového jazyka alebo procesora. Zahŕňa najmä optimalizácie a generovanie cieľového kódu.

Rozhranie medzi front-endom a back-endom tvorí medzikód. Výhodou tohto rozdelenia je modularita: možno kombinovať rôzne front-endy pre rôzne zdrojové jazyky a rôzne back-endy pre rôzne cieľové architektúry.

1.5 Vzťah k teórii jazykov

Kompilátory úzko súvisia s teóriou formálnych jazykov:

- lexikálna analýza používa regulárne jazyky a konečné automaty,
- syntaktická analýza používa bezkontextové gramatiky,
- sémantická analýza rieši vlastnosti, ktoré bezkontextová gramatika zvyčajne nezachytáva.

Syntax určuje, ako je program zapísaný. Sémantika určuje, čo program znamená. Napríklad výraz môže mať správnu syntax, ale nesprávnu sémantiku, ak obsahuje nedeklarovanú premennú alebo typovo nesprávnu operáciu.

1.6 Kompilátor, interpreter a JIT

Klasický kompilátor Klasický kompilátor preloží program pred jeho spustením do cieľového kódu. Výsledný program sa potom vykonáva samostatne.

Interpreter Interpreter program neprekladá celý dopredu, ale postupne ho číta a vykonáva. Výhodou je jednoduchosť a prenositeľnosť, nevýhodou spravidla nižší výkon.

Byte-kód Niektoré jazyky sa prekladajú do byte-kódu pre abstraktný stroj. Príkladom je Java Bytecode alebo CIL v platforme .NET.

JIT kompilátor JIT, teda *just-in-time* kompilátor, prekladá byte-kód alebo inú medzireprezentáciu do strojového kódu počas behu programu. Kombinuje výhody interpretácie a kompilácie.

1.7 Netypické kompilátory

Medzi netypické kompilátory patria:

- *source-to-source* kompilátory, ktoré prekladajú jeden vyšší jazyk do iného vyššieho jazyka,
- cross-kompilátory, ktoré generujú kód pre inú platformu, než na ktorej bežia,
- dekompilátory, ktoré sa snažia preložiť nižší jazyk do vyššieho,
- kompilátory do byte-kódu pre virtuálne stroje.

1.8 Použitie techník kompilátorov

Techniky kompilátorov sa používajú aj mimo klasických kompilátorov:

- syntax-highlighting editory,
- pretty printery,
- statické kontroly programov,
- interpretery,
- formátory textu, napríklad $\text{\LaTeX}$,
- interpretery dotazovacích jazykov, napríklad SQL,
- spracovanie štruktúrovaných dokumentov, napríklad XML a JSON.

1.9 Nástroje pre tvorbu kompilátorov

Flex Flex je generátor lexikálnych analyzátorov. Na vstupe má regulárne výrazy opisujúce tokeny a akcie, ktoré sa vykonajú pri ich rozpoznaní. Výstupom je scanner.

Bison Bison je generátor syntaktických analyzátorov. Na vstupe má bezkontextovú gramatiku a sémantické akcie. Často sa používa spolu s Flexom.

ANTLR ANTLR je nástroj na generovanie lexerov a parserov pre rôzne cieľové jazyky. Používa sa na implementáciu parserov programovacích jazykov, doménovo špecifických jazykov a štruktúrovaných formátov.

1.10 Jednoprechodový kompilátor

Jednoprechodový kompilátor spracuje vstupný program v jednom prechode. Počas čítania vstupu vykonáva analýzu a priamo generuje medzikód alebo cieľový kód.

Výhody:

- jednoduchá implementácia,
- nízke pamäťové nároky,
- rýchly preklad.

Nevýhody:

- obmedzené možnosti optimalizácie,
- problém s doprednými odkazmi,
- nevhodnosť pre zložité jazyky.

1.11 Rekurzívny zostup

Rekurzívny zostup je top-down metóda syntaktickej analýzy. Každému neterminálu gramatiky zodpovedá jedna procedúra. Parser postupuje od koreňa syntaktického stromu k listom a podľa aktuálneho tokenu rozhoduje, ktoré pravidlo použije.

Pre gramatiku:

$$\begin{aligned}expr &\rightarrow term\ expr', \\expr' &\rightarrow +term\ expr' \mid \varepsilon, \\term &\rightarrow factor\ term', \\term' &\rightarrow *factor\ term' \mid \varepsilon, \\factor &\rightarrow id \mid number \mid (expr)\end{aligned}$$

môžeme napísať procedúry:

`parseExpr, parseExprRest, parseTerm, parseTermRest, parseFactor.`

Rekurzívny zostup je vhodný najmä pre LL gramatiky. Nevie však priamo spracovať ľavorekuzívne gramatiky, preto ich treba upraviť.

1.12 Krátke zhrnutie na štátnice

Kompilátor prekladá program zo zdrojového jazyka do cieľového jazyka. Skladá sa z lexikálnej, syntaktickej a sémantickej analýzy, generovania medzikódu, optimalizácie a generovania cieľového kódu. Lexikálna analýza súvisí s regulárnymi jazykmi a konečnými automatmi, syntaktická analýza s bezkontextovými gramatikami. Syntax opisuje tvar programu, sémantika jeho význam. Kompilátor sa delí na front-end závislý od zdrojového jazyka a back-end závislý od cieľovej architektúry. Výstupom môže byť iný jazyk, byte-kód, assembler alebo strojový kód. Alternatívou je interpreter a kombináciou prístupov je JIT. Techniky kompilátorov sa používajú aj pri statickej kontrole, formátovaní, SQL, XML a JSON. Na tvorbu kompilátorov sa používajú nástroje ako Flex, Bison a ANTLR. Pre jednoduché jazyky možno použiť jednoprechodový kompilátor s rekurzívnym zostupom.

2 Lexikálna analýza

2.1 Úloha lexikálnej analýzy

Lexikálna analýza je prvá fáza kompilátora. Jej primárnou úlohou je čítať znaky zdrojového programu a prekladať ich na postupnosť tokenov, ktorú ďalej spracúva syntaktická analýza.

Okrem toho lexikálny analyzátor:

- odstraňuje komentáre a biely priestor,
- môže ukladať informácie do tabuľky symbolov,

- priraduje chybovým hláseniam pozíciu v zdrojovom programe,
- pracuje na úrovni regulárnych jazykov.

2.2 Dôvody oddelenia lexikálnej a syntaktickej analýzy

Lexikálna a syntaktická analýza sa oddelujú najmä z týchto dôvodov:

1. **Zjednodušenie návrhu.** Parser nemusí riešiť jednotlivé znaky, medzery, tabulátory, komentáre ani konkrétne tvary identifikátorov a čísel.
2. **Zvýšenie efektívnosti.** Lexikálny analyzátor je špecializovaný na rýchle čítanie znakov a rozpoznávanie regulárnych vzorov.
3. **Zvýšenie portability.** Pri zmene reprezentácie znakov alebo kódovania často stačí upraviť iba lexikálny analyzátor.

2.3 Token, lexéma a pattern

Token Token je kategória reťazcov so spoločným významom. Je výstupom lexikálnej analýzy a vstupom syntaktickej analýzy. Z pohľadu syntaktickej analýzy je token terminál.

Príklady tokenov:

id, num, while, assign, op_plus, op_mul.

Lexéma Lexéma je konkrétna postupnosť znakov v zdrojovom programe, ktorá zodpovedá niektorému tokenu.

Pattern Pattern alebo vzor je pravidlo opisujúce množinu lexém pre daný token. Typicky je špecifikovaný regulárnym výrazom.

Napríklad:

$$id \rightarrow letter(letter \mid digit)^*,$$

$$num \rightarrow digit^+.$$

Pre vstup:

`dráha := počiatok + čas * 60`

môže byť výstup lexikálnej analýzy:

`id assign id op_plus id op_mul num.`

2.4 Atribúty tokenov

Lexikálny analyzátor často nevracia iba token, ale dvojicu:

$(\text{token}, \text{atribút}).$

Atribút upresňuje konkrétnu inštanciu tokenu. Napríklad pri identifikátore môže byť atribútom smerník do tabuľky symbolov, pri čísle jeho hodnota a pri relačnom operátore konkrétny druh operátora.

Príklad:

```
while (i <= 25) j++;
```

lexéma	token	atribút
while	<i>while</i>	–
{	<i>leftPpar</i>	–
i	<i>id</i>	smerník do tabuľky symbolov
<=	<i>relop</i>	<i>LE</i>
25	<i>num</i>	hodnota alebo smerník
}	<i>rightPpar</i>	–
j	<i>id</i>	smerník do tabuľky symbolov
++	<i>op_inc</i>	–
;	<i>semiColon</i>	–

2.5 Spolupráca s tabuľkou symbolov

Pri rozpoznaní identifikátora lexikálny analyzátor skontroluje, či už existuje v tabuľke symbolov. Ak áno, vráti smerník na príslušný záznam. Ak nie, vytvorí nový záznam.

Typické operácie:

lookup(s)

vráti index záznamu pre reťazec *s* alebo informáciu, že záznam neexistuje.

insert(s,t)

vloží nový záznam pre reťazec *s* a token *t*.

Rezervované slová môžu byť v tabuľke symbolov predvyplnené, čo zjednodušuje rozlišovanie identifikátorov a kľúčových slov.

2.6 Odsadenie ako syntaktická konštrukcia

Vo väčšine jazykov je biely priestor iba oddelovač tokenov. V niektorých jazykoch však odsadenie určuje syntaktickú štruktúru programu. Typickým príkladom je Python.

Napríklad:

```
if x > 0:
    y = 1
    z = 2
    w = 3
```

Lexikálny analyzátor v takom prípade nemôže odsadenie ignorovať. Musí generovať špeciálne tokeny, napríklad:

NEWLINE, INDENT, DEDENT.

Tak sa zmena odsadenia prevedie na explicitné tokeny, ktoré parser spracuje podobne ako zátvorky blokov.

2.7 Regulárne jazyky a regulárne výrazy

Lexikálna analýza pracuje na úrovni regulárnych jazykov. Regulárne jazyky sú jazyky, ktoré možno opísať regulárnymi výrazmi a rozpoznať konečnými automatmi.

Základné pravidlá regulárnych výrazov:

1. ε je regulárny výraz označujúci jazyk $\{\varepsilon\}$.
2. Ak $a \in \Sigma$, potom a je regulárny výraz označujúci jazyk $\{a\}$.
3. Ak r a s sú regulárne výrazy, potom:

$$r|s$$

označuje zjednotenie,

$$rs$$

označuje konkatenáciu,

$$r^*$$

označuje Kleeneho uzáver.

Používané skratky:

$$\begin{aligned} r^+ & \text{ jeden alebo viac výskytov,} \\ r? & \text{ nula alebo jeden výskyt,} \\ [a - z] & \text{ trieda znakov.} \end{aligned}$$

Príklady regulárnych definícií:

$$\begin{aligned} letter & \rightarrow [A - Za - z], \\ digit & \rightarrow [0 - 9], \\ id & \rightarrow letter(letter | digit)^*, \\ num & \rightarrow digit^+, \\ relop & \rightarrow <|<=|=|<>|>|>= . \end{aligned}$$

2.8 Konečné automaty

Konečný automat je päťica:

$$A = (Q, \Sigma, \delta, q_0, F),$$

kde:

- Q je konečná množina stavov,

- Σ je vstupná abeceda,
- δ je prechodová funkcia,
- $q_0 \in Q$ je počiatočný stav,
- $F \subseteq Q$ je množina akceptačných stavov.

Konečné automaty rozpoznávajú práve regulárne jazyky.

2.8.1 Deterministický konečný automat

V deterministickom konečnom automate je pre každý stav a vstupný symbol určený najviac jeden nasledujúci stav. Prechodová funkcia má tvar:

$$\delta : Q \times \Sigma \rightarrow Q.$$

Simulácia DKA:

```

q := q0;
kým nie je koniec vstupu:
    q := δ(q, c);
ak q ∈ F, akceptuj, inak zamietni.
```

2.8.2 Nedeterministický konečný automat

V nedeterministickom konečnom automate môže pre jeden stav a vstupný symbol existovať viacero možných nasledujúcich stavov. Môžu byť povolené aj ε -prechody.

Prechodová funkcia má tvar:

$$\delta : Q \times \Sigma \rightarrow 2^Q.$$

NKA akceptuje vstup vtedy, ak existuje aspoň jedna cesta, ktorá po prečítaní vstupu skončí v akceptačnom stave.

NKA a DKA sú ekvivalentné z hľadiska vyjadrovacej sily: rozpoznávajú práve regulárne jazyky.

2.9 Prechod od regulárnych výrazov k DKA

Typický postup tvorby lexikálneho analyzátora je:

regulárne výrazy $\rightarrow$ NKA $\rightarrow$ DKA $\rightarrow$ minimalizovaný DKA $\rightarrow$ implementácia.

2.9.1 Thompsonova konštrukcia

Thompsonova konštrukcia vytvára NKA z regulárneho výrazu podľa jeho štruktúry:

- pre ε sa vytvorí automat s ε -prechodom,
- pre symbol a sa vytvorí automat s prechodom označeným a ,

- pre $s|t$ sa vytvorí vetvenie pomocou ε -prechodov,
- pre st sa spojí automat pre s s automatom pre t ,
- pre s^* sa pridajú ε -prechody umožňujúce opakovanie aj vynechanie výrazu.

2.9.2 Subset konštrukcia DKA z NKA

Jeden stav DKA zodpovedá množine stavov NKA. Používajú sa operácie:

$$\varepsilon\text{-closure}(T)$$

a

$$\text{move}(T, a).$$

Algoritmus:

```

Dstates := { $\varepsilon$ -closure( $q_0$ )};
kým existuje neoznačený stav  $q \in Dstates$  :
    označ  $q$ ;
    pre každý symbol  $a$  :
         $U := \varepsilon\text{-closure}(\text{move}(q, a))$ ;
        ak  $U \notin Dstates$ , pridaj  $U$ ;
         $Dtran[q, a] := U$ .

```

Výsledný DKA akceptuje ten istý jazyk ako pôvodný NKA.
 Výpočet

ε

-closure

Výpočet $\varepsilon\text{-closure}(T)$:

```

vlož všetky stavy z  $T$  do zásobníka;
inicializuj  $\varepsilon\text{-closure}(T)$  na  $T$ ;
while zásobník nie je prázdny do begin
    vyber  $q$ , vrchný symbol zo zásobníka;
    for každý stav  $s$ , do ktorého sa dá dostať z  $q$  na  $\varepsilon$  do begin
        if  $s \notin \varepsilon\text{-closure}(T)$  then begin
            pridaj  $s$  do  $\varepsilon\text{-closure}(T)$ ;
            vlož  $s$  na vrch zásobníka;
        end;
    end;
end.

```

2.9.3 Zložitosť

Priama simulácia NKA:

$$\text{priestor } O(|r|), \quad \text{čas } O(|r| \cdot |x|),$$

kde $|r|$ je dĺžka regulárneho výrazu a $|x|$ je dĺžka vstupu.

Zostrojenie a simulácia DKA:

priestor $O(2^{|r|})$, čas simulácie $O(|x|)$.

2.10 Prechodové diagramy

Prechodový diagram je graf reprezentujúci rozpoznávanie tokenov. Vrcholy sú stavy a hrany sú označené vstupnými znakmi alebo triedami znakov.

Napríklad pre relačné operátory možno rozpoznávať:

$<, <=, <>, =, >, >=$.

Ak automat prečíta znak, ktorý už nepatrí do aktuálnej lexémy, môže ho byť potrebné vrátiť späť na vstup. Takýto prípad sa označuje ako lookahead.

2.11 Vstupné a výstupné rozhranie lexikálnej analýzy

2.11.1 Vstupný buffer

Lexikálny analyzátor je jediná fáza, ktorá číta vstup znak po znaku. Pre efektívnosť sa používa vstupný buffer. Pri dvojbufferovej schéme sa používajú dva smerníky:

lexeme_beginning

a

forward.

Smerník *lexeme_beginning* označuje začiatok aktuálnej lexémy a smerník *forward* sa posúva dopredu, kým sa lexéma nerozpozna.

2.11.2 Výstupný buffer

Medzi lexikálnou a syntaktickou analýzou je výstupný buffer tokenov. Lexikálny analyzátor je producent tokenov a parser je ich konzument. V praxi má buffer často veľkosť 1 a parser si volá lexikálny analyzátor, keď potrebuje ďalší token.

2.12 Vyhľadávanie v texte: Aho-Corasick

Algoritmus Aho-Corasick slúži na vyhľadávanie všetkých výskytov vzorov z konečnej množiny patternov v texte.

Pre množinu vzorov:

$$P = \{p_1, p_2, \dots, p_k\}$$

algoritmus najprv vytvorí prefixový strom. Potom pridá fail hrany, ktoré určujú, kam sa má automat presunúť, ak nie je možné pokračovať aktuálnou hranou. Vrcholy zodpovedajúce celým vzorom sa označia ako koncové stavy.

Príklad množiny vzorov:

$$P = \{\text{elseif}, \text{else}, \text{end}, \text{for}, \text{then}\}.$$

Časová zložitosť algoritmu je lineárna vzhľadom na dĺžku vzorov, dĺžku prehľadávaného textu a počet výskytov:

$$O(\text{dĺžka vzorov} + \text{dĺžka textu} + \text{počet výskytov}).$$

Na rozdiel od lexikálnej analýzy hľadané reťazce nemusia byť oddelené bielym priestorom a môžu sa aj prekrývať.

2.13 Oprava chýb a zotavenie pri lexikálnej analýze

Lexikálna chyba nastane vtedy, keď postupnosť znakov na vstupe nezodpovedá žiadnemu patternu tokenu.

Možnosti zotavenia:

1. vymazanie znakov zo vstupu, kým sa nenájde reťazec zodpovedajúci niektorému patternu,
2. vloženie znakov navyše,
3. výmena nesprávneho znaku za správny,
4. výmena susedných znakov.

V praxi sa často použije jednoduchá stratégia: lexikálny analyzátor vypíše chybové hlásenie s pozíciou chyby, preskočí problematické znaky po najbližší rozpoznateľný token alebo oddelovač a pokračuje v analýze.

2.14 Krátke zhrnutie

Lexikálna analýza číta znaky zdrojového programu a vytvára tokeny pre syntaktickú analýzu. Token je kategória reťazcov, lexéma je konkrétny reťazec a pattern je pravidlo opisujúce lexémy daného tokenu. Tokeny sa opisujú regulárnymi výrazmi, ktoré možno previesť na NKA a následne na DKA. DKA sa potom efektívne simuluje pomocou prechodovej tabuľky. Lexikálna analýza sa oddeľuje od syntaktickej kvôli jednoduchšiemu návrhu, efektívnosti a portabilite. V jazykoch ako Python musí LA spracovať odsadenie ako syntaktickú konštrukciu generovaním tokenov ako `INDENT` a `DEDENT`. Pri chybách sa používa zotavenie, napríklad preskočenie neplatných znakov alebo lokálne opravy.

3 Syntaktická analýza zhora-nadol

3.1 Úloha syntaktickej analýzy

Syntaktická analýza číta postupnosť tokenov generovaných lexikálnym analyzátorom a overuje, či táto postupnosť môže byť generovaná bezkontextovou gramatikou vstupného jazyka. Výstupom je strom odvodenia, syntaktický strom alebo abstraktný syntaktický strom.

Parser tiež hlási syntaktické chyby a snaží sa z nich zotaviť tak, aby bolo možné pokračovať v spracovaní vstupu.

3.2 Bezkontextová gramatika

Bezkontextová gramatika je štvorica

$$G = (N, T, P, S),$$

kde:

- N je konečná množina neterminálov,
- T je konečná množina terminálov,
- P je konečná množina pravidiel tvaru

$$A \rightarrow \alpha,$$

kde $A \in N$ a $\alpha \in (N \cup T)^*$,

- $S \in N$ je počiatočný neterminál.

Terminály sú z pohľadu syntaktickej analýzy tokeny posielať lexikálnym analyzátorom. Neterminály reprezentujú syntaktické konštrukcie, napríklad výraz, príkaz alebo blok.

3.3 Odvodenie

Krok odvodenia má tvar

$$\alpha A \beta \Rightarrow \alpha \gamma \beta,$$

ak existuje pravidlo

$$A \rightarrow \gamma.$$

Odvodenie z počiatočného symbolu zapisujeme

$$S \Rightarrow^* \alpha.$$

Ak sa vetná forma skladá iba z terminálov, ide o slovo generované gramatikou.

Pri odvodení rozlišujeme:

- ľavé krajné odvodenie, kde sa vždy prepisuje najľavejší neterminál,
- pravé krajné odvodenie, kde sa vždy prepisuje najpravejší neterminál.

Pri parsovaní zhora-nadol sa konštruuje ľavé krajné odvodenie.

3.4 Strom odvodenia

Strom odvodenia je grafická reprezentácia odvodenia. Vnútorne uzly zodpovedajú aplikáciám pravidiel a listy zodpovedajú terminálom alebo ε .

Strom odvodenia neznázorňuje poradie aplikovania pravidiel. Jednému stromu odvodenia zodpovedá práve jedno ľavé krajné odvodenie a práve jedno pravé krajné odvodenie.

3.5 Abstraktný syntaktický strom

Abstraktný syntaktický strom, skrátene AST, je zjednodušená verzia syntaktického stromu. Zachováva významovú štruktúru programu, ale odstraňuje technické detaily gramatiky, ako sú pomocné neterminály, zátvorky alebo oddeľovače.

Napríklad výraz

$$a + b * c$$

môže byť reprezentovaný abstraktným syntaktickým stromom

$$+(a, *(b, c)).$$

3.6 Nejednoznačnosť gramatiky

Gramatika je nejednoznačná, ak existuje aspoň jedno slovo, pre ktoré existujú dva rôzne stromy odvodenia.

Príklad:

$$E \rightarrow E + E \mid E * E \mid id.$$

Pre vstup

$$id + id * id$$

existujú dve možné interpretácie:

$$(id + id) * id$$

a

$$id + (id * id).$$

Nejednoznačnosť spôsobuje konflikty pri parsovaní. Niekedy ju možno odstrániť úpravou gramatiky, napríklad zavedením priorít a asociativity operátorov:

$$\begin{aligned} E &\rightarrow E + T \mid T, \\ T &\rightarrow T * F \mid F, \\ F &\rightarrow id \mid num \mid (E). \end{aligned}$$

3.7 Eliminácia ľavej rekurzie

Gramatika je ľavorekuzívna, ak pre nejaký neterminál A platí

$$A \Rightarrow^+ A\alpha.$$

Parsery zhora-nadol nevedia priamo spracovať ľavorekuzívne gramatiky, pretože by mohlo dôjsť k nekonečnej rekurzii bez spotrebovania vstupu.

3.7.1 Eliminácia priamej ľavej rekurzie

Ak máme pravidlá

$$A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n,$$

kde žiadne β_i nezačína neterminálom A , nahradíme ich pravidlami:

$$A \rightarrow \beta_1 A' \mid \cdots \mid \beta_n A',$$

$$A' \rightarrow \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

Príklad:

$$E \rightarrow E + T \mid T$$

prepíšeme na:

$$E \rightarrow TE',$$

$$E' \rightarrow +TE' \mid \varepsilon.$$

3.7.2 Eliminácia všeobecnej ľavej rekurzie

Neterminály usporiadame v poradí

$$A_1, A_2, \dots, A_n.$$

Potom pre každé $i = 1, \dots, n$ a každé $j = 1, \dots, i - 1$ nahradíme každé pravidlo tvaru

$$A_i \rightarrow A_j \gamma$$

pravidlami

$$A_i \rightarrow \delta_1 \gamma \mid \cdots \mid \delta_k \gamma,$$

kde

$$A_j \rightarrow \delta_1 \mid \cdots \mid \delta_k$$

sú všetky aktuálne pravidlá pre A_j . Potom eliminujeme priamu ľavú rekurziu medzi pravidlami pre A_i .

3.8 Ľavá faktorizácia

Ľavá faktorizácia sa používa pri prediktívnom parsovaní vtedy, keď viaceré pravidlá pre ten istý neterminál začínajú rovnakým prefixom.

Pravidlá

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

nahradíme pravidlami

$$A \rightarrow \alpha A',$$

$$A' \rightarrow \beta_1 \mid \beta_2.$$

Príklad:

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$$

prepíšeme na:

$$S \rightarrow \text{if } E \text{ then } S S',$$

$$S' \rightarrow \text{else } S \mid \varepsilon.$$

3.9 Parovanie zhora-nadol

Pri parovaní zhora-nadol sa strom odvodenia konštruje od koreňa k listom. Parser začína počiatočným neterminálom a snaží sa odvodiť vstupný reťazec tokenov. Konštruje sa ľavé krajné odvodenie.

3.10 Rekurzívny zostup

Rekurzívny zostup je všeobecná metóda parovania zhora-nadol. Pre každý neterminál gramatiky existuje jedna procedúra.

Všeobecná procedúra pre neterminál A má tvar:

```
void A() {  
    vyber pravidlo  $A \rightarrow X_1 X_2 \dots X_k$ ;  
    for  $i = 1$  to  $k$  do  
        if  $X_i$  je neterminál, zavolaj procedúru  $X_i()$ ;  
        else if  $X_i$  sa rovná aktuálnemu vstupnému symbolu,  
            posuň sa na ďalší vstupný symbol;  
        else nastala syntaktická chyba;  
}
```

Všeobecný rekurzívny zostup môže používať backtracking. Pri ľavorekurzívnych gramatikách však môže dôjsť k nekonečnému cykleniu.

3.11 Prediktívne parovanie

Prediktívne parovanie je špeciálny prípad rekurzívneho zostupu, pri ktorom nie je potrebný backtracking. Parser sa musí vedieť rozhodnúť, ktoré pravidlo použiť, na základe aktuálneho neterminálu a lookahead symbolu.

3.12 Množiny FIRST a FOLLOW

3.12.1 FIRST

Množina

$$FIRST(\alpha)$$

obsahuje terminály, ktorými môže začínať reťazec odvodený z α . Ak

$$\alpha \Rightarrow^* \varepsilon,$$

potom

$$\varepsilon \in FIRST(\alpha).$$

3.12.2 FOLLOW

Množina

$$FOLLOW(A)$$

obsahuje terminály, ktoré sa môžu vyskytnúť bezprostredne za neterminálom A v nejakej vetnej forme. Formálne, terminál a patrí do $FOLLOW(A)$, ak existuje odvodenie

$$S \Rightarrow^* \alpha A a \beta.$$

Ak A môže byť najpravejší symbol vo vetnej forme, potom

$$\$ \in FOLLOW(A).$$

3.13 Výpočet FIRST

Pre každý symbol X :

1. Ak X je terminál, potom

$$FIRST(X) = \{X\}.$$

2. Ak X je neterminál a existuje pravidlo $X \rightarrow \varepsilon$, pridaj ε do $FIRST(X)$.
3. Ak X je neterminál a existuje pravidlo

$$X \rightarrow Y_1 Y_2 \dots Y_k,$$

pridaj a do $FIRST(X)$, ak $a \in FIRST(Y_i)$ a

$$\varepsilon \in FIRST(Y_1), \dots, \varepsilon \in FIRST(Y_{i-1}).$$

Pridaj ε do $FIRST(X)$, ak

$$\varepsilon \in FIRST(Y_1), \dots, \varepsilon \in FIRST(Y_k).$$

Pre reťazec $X_1 X_2 \dots X_n$ pridávame

$$FIRST(X_i) - \{\varepsilon\}$$

ak

$$\varepsilon \in FIRST(X_1), \dots, \varepsilon \in FIRST(X_{i-1}).$$

Ak všetky symboly môžu odvodiť ε , pridáme aj ε .

3.14 Výpočet FOLLOW

1. Pridaj $\$$ do $FOLLOW(S)$.
2. Ak existuje pravidlo

$$A \rightarrow \alpha B \beta,$$

pridaj

$$FIRST(\beta) - \{\varepsilon\}$$

do $FOLLOW(B)$.

3. Ak existuje pravidlo

$$A \rightarrow \alpha B$$

alebo pravidlo

$$A \rightarrow \alpha B \beta$$

také, že

$$\varepsilon \in FIRST(\beta),$$

pridaj všetko z $FOLLOW(A)$ do $FOLLOW(B)$.

3.15 LL(1) gramatiky

LL(1) znamená:

- prvé L: vstup sa číta zľava doprava,
- druhé L: konštruuje sa ľavé krajné odvodenie,
- 1: lookahead je obmedzený na jeden vstupný symbol.

Gramatika je LL(1), ak pre každú dvojicu rôznych pravidiel

$$A \rightarrow \alpha \mid \beta$$

platí:

$$FIRST(\alpha) \cap FIRST(\beta) = \emptyset.$$

Ak

$$\varepsilon \in FIRST(\beta),$$

potom musí platiť

$$FIRST(\alpha) \cap FOLLOW(A) = \emptyset.$$

Ak

$$\varepsilon \in FIRST(\alpha),$$

potom musí platiť

$$FIRST(\beta) \cap FOLLOW(A) = \emptyset.$$

Nejednoznačné a ľavorekuzívne gramatiky nie sú LL(1).

3.16 LL(k)

LL(k) je zovšeobecnenie LL(1), kde parser používa k symbolov lookahead. Vstup sa stále číta zľava doprava a konštruuje sa ľavé krajné odvodenie.

Platí:

$$LL(1) \subseteq LL(2) \subseteq \dots \subseteq LL(k).$$

Vyššie k umožňuje rozlíšiť viac situácií, ale implementácia parsera je zložitejšia.

3.17 Konštrukcia parsovacej tabuľky

Pre každé pravidlo

$$A \rightarrow \alpha$$

vykonáme:

1. Pre každý terminál

$$a \in FIRST(\alpha)$$

pridáme pravidlo $A \rightarrow \alpha$ do položky

$$M[A, a].$$

2. Ak

$$\varepsilon \in FIRST(\alpha),$$

pridáme pravidlo $A \rightarrow \alpha$ do

$$M[A, b]$$

pre každý terminál

$$b \in FOLLOW(A).$$

3. Ak

$$\varepsilon \in FIRST(\alpha)$$

a

$$\$ \in FOLLOW(A),$$

pridáme pravidlo $A \rightarrow \alpha$ do

$$M[A, \$].$$

4. Každý nedefinovaný záznam označíme ako chybu.

3.18 Algoritmus parsovania pomocou zásobníka

a := prvý symbol vstupu;

X := vrchný symbol zásobníka;

while $X \neq \$$ **do**

if $X = a$ **then**

 vyber X zo zásobníka a priraď do a ďalší symbol vstupu;

else if X je terminál **then**

$error()$;

else if $M[X, a]$ je chybové políčko **then**

$error()$;

else if $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$ **then**

 daj na výstup pravidlo $X \rightarrow Y_1 Y_2 \dots Y_k$;

 vyber X zo zásobníka;

 vloď $Y_k, Y_{k-1}, \dots, Y_1$ do zásobníka;

X := vrchný symbol zásobníka.

3.19 Všeobecné metódy pre bezkontextové jazyky

Univerzálne metódy syntaktickej analýzy fungujú pre všetky bezkontextové gramatiky, ale sú menej efektívne. Typicky majú zložitosť $O(n^3)$.

3.19.1 Cocke-Younger-Kasami

Algoritmus Cocke-Younger-Kasami, skrátene CYK, je dynamický algoritmus pre bezkontextové gramatiky, zvyčajne v Chomského normálnej forme. Vypĺňa tabuľku, kde položky určujú, ktoré neterminály môžu odvodiť konkrétny podreťazec vstupu.

Ak počiatočný symbol S dokáže odvodiť celý vstupný reťazec, vstup patrí do jazyka.

3.19.2 Earleyho algoritmus

Earleyho algoritmus je všeobecný algoritmus pre syntaktickú analýzu bezkontextových gramatík. Pracuje s položkami tvaru

$$[A \rightarrow \alpha \bullet \beta, i],$$

kde bodka označuje aktuálnu pozíciu v pravej strane pravidla a i označuje pozíciu vo vstupe, kde sa rozpoznávanie začalo.

Používa operácie:

- predictor,
- scanner,
- completer.

Všeobecná časová zložitosť je $O(n^3)$.

3.20 Oprava chýb a zotavenie

Syntaktická chyba nastane vtedy, keď postupnosť tokenov nezodpovedá pravidlám gramatiky.

Ciele spracovania chýb:

- jasné a presné hlásenie chýb,
- rýchle zotavenie,
- minimálne spomalenie korektných programov.

3.20.1 Zotavenie v móde paniky

Zvolí sa množina synchronizačných tokenov, napríklad

$$;, \text{ end}, \}.$$

Pri detekovaní chyby sa vynecháva vstup, kým sa nenájde synchronizačný token. Pri LL parseri môžu byť synchronizačnými tokenmi napríklad symboly z množiny $FOLLOW(A)$ pre aktuálny neterminál A .

3.20.2 Zotavenie na úrovni frázy

Parser vykoná lokálnu korekciu, napríklad vloženie, vymazanie alebo nahradenie tokenu. Pri LL parseroch možno prázdne záznamy parsovej tabuľky vyplniť smerníkmi na procedúry ošetrojúce konkrétnu chybu.

3.20.3 Chybové pravidlá

Gramatika sa rozšíri o pravidlá generujúce časté chybné konštrukcie. Keď parser použije chybové pravidlo, vygeneruje diagnostické hlásenie.

3.20.4 Globálne korekcie

Globálne korekcie hľadajú najvýhodnejšiu opravu vstupu, napríklad minimálnu postupnosť vložení, vymazaní alebo náhrad tokenov. Sú zaujímavé teoreticky, ale drahé z hľadiska času a pamäte.

3.21 Zhrnutie

Syntaktická analýza zhora-nadol konštruuje strom odvodenia od koreňa k listom a zodpovedá ľavému krajnému odvodeniu. Používa bezkontextové gramatiky. Pre prediktívne parsovanie treba gramatiku často upraviť: odstrániť nejednoznačnosť, eliminovať ľavú rekúziu a vykonať ľavú faktorizáciu. Rekurzívny zostup používa jednu procedúru pre každý neterminál. LL(1) parser číta vstup zľava doprava, konštruuje ľavé krajné odvodenie a používa jeden lookahead symbol. Pri konštrukcii LL(1) parsera sa používajú množiny FIRST a FOLLOW. Univerzálne metódy ako Earleyho algoritmus a CYK fungujú pre všetky bezkontextové gramatiky, ale majú typicky zložitosť $O(n^3)$. Pri chybách sa používajú stratégie ako mód paniky, fráza-level zotavenie, chybové pravidlá a globálne korekcie.

4 Syntaktická analýza zdola-nahor

4.1 Základná charakteristika

Syntaktická analýza zdola-nahor konštruuje strom odvodenia od listov ku koreňu. Parser začína vstupnou postupnosťou tokenov a postupnými redukciami sa ju snaží previesť na počiatočný symbol gramatiky.

V každom redukčnom kroku sa podreťazec zodpovedajúci pravej strane nejakého pravidla nahradí neterminálom z ľavej strany pravidla.

Cieľom je získať pravé krajné odvodenie zapísané odzadu.

4.2 Bezkontextová gramatika

Bezkontextová gramatika je štvorica

$$G = (N, T, P, S),$$

kde:

- N je konečná množina neterminálov,
- T je konečná množina terminálov,
- P je konečná množina pravidiel tvaru

$$A \rightarrow \alpha,$$

kde $A \in N$ a $\alpha \in (N \cup T)^*$,

- S je počiatočný neterminál.

Terminály sú tokeny prichádzajúce z lexikálneho analyzátora. Neterminály reprezentujú syntaktické konštrukcie ako výraz, príkaz alebo blok.

4.3 Strom odvodenia a abstraktný syntaktický strom

Strom odvodenia je grafická reprezentácia aplikácií pravidiel gramatiky. Pri parsovaní zdola-nahor sa strom konštruuje od listov ku koreňu.

Abstraktný syntaktický strom, skrátene AST, je zjednodušená reprezentácia syntaktickej štruktúry programu. Odstraňuje technické detaily gramatiky a zachováva významovú štruktúru programu.

Napríklad výraz

$$a + b * c$$

môže mať abstraktný syntaktický strom

$$+(a, *(b, c)).$$

4.4 Nejednoznačnosť gramatiky

Gramatika je nejednoznačná, ak existuje slovo, ktoré má dva rôzne stromy odvodenia.

Príklad:

$$E \rightarrow E + E \mid E * E \mid id.$$

Pre vstup

$$id + id * id$$

existujú dve možné interpretácie:

$$(id + id) * id$$

a

$$id + (id * id).$$

Nejednoznačnosť sa často odstraňuje zavedením priorít a asociativity operátorov:

$$E \rightarrow E + T \mid T,$$

$$T \rightarrow T * F \mid F,$$

$$F \rightarrow id \mid (E).$$

4.5 Handle

Rukoväť, teda handle, je podreťazec vetnej formy, ktorý zodpovedá pravej strane nejakého pravidla a ktorého redukcia predstavuje jeden krok pravého krajného odvodenia odzadu.

Formálne, handle vetnej formy γ je pravidlo

$$A \rightarrow \beta$$

a pozícia v γ , kde sa β nachádza, pričom nahradením β neterminálom A získame predchádzajúcu pravú vetnú formu.

Postupná redukcia má tvar

$$w = \gamma_n \leftarrow_{rm} \gamma_{n-1} \leftarrow_{rm} \cdots \leftarrow_{rm} \gamma_1 \leftarrow_{rm} \gamma_0 = S.$$

4.6 Životaschopný prefix

Životaschopný prefix je prefix pravej vetnej formy, ktorý nepresahuje pravý koniec najpravejšej handle vetnej formy. LR parsery používajú deterministický konečný automat na rozpoznávanie životaschopných prefixov.

4.7 Shift-reduce parsovanie

Shift-reduce parsovanie je všeobecná metóda syntaktickej analýzy zdola-nahor. Parser používa zásobník a vstupný buffer.

Základné akcie parsera:

1. **shift**: vstupný symbol sa vloží na zásobník,
2. **reduce**: handle na vrchu zásobníka sa redukuje podľa pravidla gramatiky,
3. **accept**: úspešné ukončenie parsovania,
4. **error**: syntaktická chyba.

Handle sa vždy objaví na vrchu zásobníka, nie vo vnútri zásobníka.

4.8 Konflikty pri shift-reduce parsovaní

4.8.1 Shift/reduce konflikt

Shift/reduce konflikt vzniká vtedy, keď parser nevie, či má ešte vykonať shift, alebo už redukovať.

Typický príklad:

$$\begin{aligned} stmt &\rightarrow \textit{if expr then stmt}, \\ stmt &\rightarrow \textit{if expr then stmt else stmt}, \\ stmt &\rightarrow \textit{other}. \end{aligned}$$

4.8.2 Reduce/reduce konflikt

Reduce/reduce konflikt vzniká vtedy, keď parser nevie, ktorým z viacerých možných pravidiel má redukovať.

Deterministický shift-reduce parser sa musí vedieť jednoznačne rozhodnúť na základe obsahu zásobníka a k vstupných symbolov. Tak vznikajú LR(k) gramatiky.

4.9 Operátorovo-precedenčné parsovanie

Operátorovo-precedenčné parsovanie je špeciálny prípad shift-reduce parsovania vhodný najmä pre aritmetické výrazy.

Operátorovo-precedenčné gramatiky sú podmnožinou LR gramatík. Majú obmedzenia:

- pravá strana pravidla nemôže byť ε ,
- neterminály na pravej strane pravidla sa nesmú vyskytovať priamo vedľa seba.

4.9.1 Precedenčné relácie

Definujú sa tri disjunktné precedenčné relácie:

$$ab$$

znamená, že a má menšiu precedenciu ako b ,

$$a \doteq b$$

znamená, že a má rovnakú precedenciu ako b ,

$$ab$$

znamená, že a má väčšiu precedenciu ako b .

Tieto relácie slúžia na ohraničenie handle:

- označuje začiatok handle,
- $\doteq$ označuje vnútro handle,
- označuje koniec handle.

4.9.2 Lokalizácia handle

Postup lokalizácie handle:

1. Prechádzaj vstupným reťazcom, kým nenájdeš prvé $.$
 2. Vráť sa doľava cez všetky $\doteq$, kým nenájdeš $.$
 3. Handle obsahuje všetko medzi nájdenými reláciami.
- Neterminály sa pri určovaní precedencie ignorujú.

4.9.3 Algoritmus operátorovo-precedenčného parsovania

Nech a je najvyšší terminál na zásobníku a b je aktuálny vstupný terminál.

```
repeat forever
  if $ je na vrchu zásobníka a vstup je $ then
    return;
  else if  $ab$  alebo  $a \doteq b$  then
    vlož  $b$  do zásobníka;
    posuň vstup na ďalší symbol;
  else if  $ab$  then
    vyberaj symboly zo zásobníka až po začiatok handle;
    redukuj nájdenú handle;
  else
    error().
```

4.9.4 Precedenčné funkcie

Precedenčnú tabuľku možno komprimovať pomocou dvoch funkcií

$$f, g : T \rightarrow Z.$$

Majú spĺňať:

$$\begin{aligned} f(a) < g(b) & \text{ ak } ab, \\ f(a) = g(b) & \text{ ak } a \doteq b, \\ f(a) > g(b) & \text{ ak } ab. \end{aligned}$$

Potom možno precedenčnú reláciu medzi a a b určiť porovnaním hodnôt $f(a)$ a $g(b)$.

4.9.5 Výhody a nevýhody

Výhody:

- jednoduchá implementácia,
- efektívne parsovanie výrazov.

Nevýhody:

- použiteľné len pre malú množinu gramatík,
- problém s tokenmi s viacerými precedenciami,
- slabšia väzba parsera na gramatiku.

4.10 LR parsovanie

LR parsovanie je efektívna technika syntaktickej analýzy zdola-nahor. Skratka $LR(k)$ znamená:

- L: vstup sa číta zľava doprava,
- R: vytvára sa pravé krajné odvodenie,
- k : používa sa k symbolov lookahead.

V praxi sa najčastejšie používajú $LR(1)$ parsery.

4.11 Model LR parsera

LR parser používa:

- vstupný buffer,
- zásobník,
- tabuľku ACTION,
- tabuľku GOTO,
- výstupnú postupnosť redukcií.

Zásobník obsahuje striedavo gramatické symboly a stavy:

$$s_0 X_1 s_1 X_2 s_2 \dots X_m s_m.$$

4.11.1 Tabuľka ACTION

Funkcia

$$action[s, a]$$

určuje akciu parsera v stave s pri vstupnom symbole a .

Možné akcie:

1. shift s' ,
2. reduce $A \rightarrow \beta$,
3. accept,
4. error.

4.11.2 Tabuľka GOTO

Funkcia

$$goto[s, A]$$

sa používa po redukcii a určuje nový stav po vložení neterminálu A .

4.12 Konfigurácie LR parsera

Konfigurácia LR parsera je dvojica

$$(s_0 X_1 s_1 X_2 s_2 \dots X_m s_m, a_i a_{i+1} \dots a_n \$).$$

Reprezentuje pravú vetnú formu

$$X_1 X_2 \dots X_m a_i a_{i+1} \dots a_n.$$

4.13 LR parsovací algoritmus

nastav ip na prvý symbol $w\$$;

repeat forever begin

nech s je stav na vrchu zásobníka a a je symbol, na ktorý ukazuje ip ;

if $action[s, a] = shift\ s'$ **then begin**

vlož a a potom s' na vrch zásobníka;

posuň ip na ďalší vstupný symbol;

end;

else if $action[s, a] = reduce\ A \rightarrow \beta$ **then begin**

vyber $2|\beta|$ symbolov zo zásobníka;

nech s' je stav navrchu zásobníka;

vlož A a potom $goto[s', A]$ na vrch zásobníka;

vypíš na výstup pravidlo $A \rightarrow \beta$;

end;

else if $action[s, a] = accept$ **then**

return;

else

$error()$;

end

4.14 LR metódy

4.14.1 SLR

SLR, teda Simple LR, je najjednoduchšia LR metóda. Používa LR(0) položky a množiny FOLLOW na rozhodovanie o redukciách.

Výhody:

- jednoduchá konštrukcia,
- malé tabuľky.

Nevýhody:

- je najslabšia z LR metód,
- môže mať konflikty aj pre gramatiky, ktoré CLR alebo LALR zvládnu.

4.14.2 CLR

CLR, teda kanonický LR parser, používa LR(1) položky tvaru

$$[A \rightarrow \alpha \bullet \beta, a],$$

kde a je lookahead symbol.

Výhody:

- najsilnejšia LR metóda,
- najmenej konfliktov.

Nevýhody:

- veľký počet stavov,
- veľké parsovacie tabuľky,
- náročná implementácia.

4.14.3 LALR

LALR, teda Lookahead LR, je kompromis medzi SLR a CLR. Vychádza z LR(1) položiek, ale zlučuje stavy s rovnakým LR(0) jadrom.

Výhody:

- tabuľky majú veľkosť podobnú SLR,
- sila je blízka CLR,
- prakticky veľmi používané.

Nevýhody:

- zlúčením stavov môžu vzniknúť konflikty.

Vzťah medzi metódami:

$$SLR \subseteq LALR \subseteq CLR.$$

4.15 Porovnanie LR a LL

Pri LR parsovaní sa pravidlo rozpoznáva podľa toho, čo už bolo odvodené z jeho pravej strany, a podľa lookahead symbolov.

Pri LL parsovaní sa pravidlo vyberá podľa neterminálu na ľavej strane a podľa lookahead symbolov na začiatku odvodenej pravej strany.

LR parser sa teda rozhoduje neskôr ako LL parser, a preto je trieda LR gramatík silnejšia než trieda LL gramatík.

4.16 Oprava chýb a zotavenie

Pri bottom-up parsovaní môžu vzniknúť najmä tieto chyby:

1. nájde sa neredukovateľná handle,
2. v parsovacej tabuľke je prázdne miesto.

Pri neredukovateľnej handle sa zlá handle vyberie zo zásobníka a namiesto redukcie sa vypíše diagnostická správa.

Pri prázdnom mieste v parsovacej tabuľke možno vykonať lokálnu korekciu na vrchu zásobníka alebo vo vstupnom bufferi, napríklad:

- vloženie tokenu,
- zmena tokenu,
- zmazanie tokenu.

Zotavenie nesmie spôsobiť zacyklenie parsera.

Všeobecné stratégie zotavenia:

- panický mód,
- zotavenie na úrovni frázy,
- chybové pravidlá,
- globálne korekcie.

4.17 LR položky a konštrukcia LR automatu

Pred konštrukciou LR parsera rozšírime gramatiku G o nový počiatočný neterminál S' a pravidlo

$$S' \rightarrow S.$$

Takto získame rozšírenú gramatiku G' .

4.17.1 LR(0) položka

LR(0) položka je pravidlo s bodkou na pravej strane. Bodka označuje, aká časť pravej strany pravidla už bola rozpoznaná. Napríklad pre pravidlo

$$A \rightarrow XYZ$$

existujú položky

$$[A \rightarrow \cdot XYZ], \quad [A \rightarrow X \cdot YZ], \quad [A \rightarrow XY \cdot Z], \quad [A \rightarrow XYZ \cdot].$$

Položka s bodkou na konci sa nazýva úplná položka.

4.17.2 Operácia closure pre LR(0)

```

function closure(I);
begin
  J := I;
  repeat
    for každú položku  $[A \rightarrow \alpha \cdot B\beta]$  v J
    and každé pravidlo  $B \rightarrow \gamma$  gramatiky G
    such that  $[B \rightarrow \cdot\gamma] \notin J$  do
      pridaj  $[B \rightarrow \cdot\gamma]$  do J;
    until do J nie je možné pridať žiadnu ďalšiu položku;
  return J;
end

```

4.17.3 Operácia goto pre LR(0)

$$\text{goto}(I, X) = \text{closure}(\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X\beta] \in I\}).$$

4.17.4 Operácia items

```

procedure items(G');
begin
  C := {closure({ $[S' \rightarrow \cdot S]$ })};
  repeat
    for každú množinu položiek I ∈ C a každý symbol gramatiky X
    such that  $\text{goto}(I, X) \neq \emptyset$  a  $\text{goto}(I, X) \notin C$  do
      pridaj  $\text{goto}(I, X)$  do C;
    until do C nie je možné pridať žiadnu ďalšiu množinu položiek;
  end

```

4.18 Konštrukcia SLR parsovacej tabuľky

Nech

$$C = \{I_0, I_1, \dots, I_n\}$$

je množina kanonických LR(0) položiek pre rozšírenú gramatiku *G'*.

Ak

$$[A \rightarrow \alpha \cdot a\beta] \in I_i$$

a

$$\text{goto}(I_i, a) = I_j,$$

kde *a* je terminál, potom

$$\text{action}[i, a] = \text{shift } j.$$

Ak

$$[A \rightarrow \alpha \cdot] \in I_i,$$

potom pre všetky

$$a \in FOLLOW(A)$$

nastavíme

$$action[i, a] = reduce\ A \rightarrow \alpha.$$

Ak

$$[S' \rightarrow S \cdot] \in I_i,$$

potom

$$action[i, \$] = accept.$$

Pre neterminály platí: ak

$$goto(I_i, A) = I_j,$$

potom

$$goto[i, A] = j.$$

4.19 LR(1) položky

LR(1) položka je dvojica LR(0) položky a lookahead symbolu:

$$[A \rightarrow \alpha \cdot \beta, a],$$

kde a je terminál alebo \$.

Lookahead sa používa pri redukcii. Podľa pravidla

$$A \rightarrow \alpha$$

redukujeme iba vtedy, ak stav obsahuje úplnú položku

$$[A \rightarrow \alpha \cdot, a]$$

a aktuálny vstupný symbol je a .

4.19.1 Operácia closure pre LR(1)

```
function closure( $I$ );  
begin  
  repeat  
    for každú položku  $[A \rightarrow \alpha \cdot B\beta, a] \in I$ ,  
    každé pravidlo  $B \rightarrow \gamma$  gramatiky  $G'$  a každý terminál  $b \in FIRST(\beta a)$   
    such that  $[B \rightarrow \cdot \gamma, b] \notin I$  do  
      pridaj  $[B \rightarrow \cdot \gamma, b]$  do  $I$ ;  
    until do  $I$  nie je možné pridať žiadnu ďalšiu položku;  
    return  $I$ ;  
end
```

4.19.2 Operácia goto pre LR(1)

```
function goto(I, X);  
begin  
  J := {[A → αX · β, a] | [A → α · Xβ, a] ∈ I};  
  return closure(J);  
end
```

4.20 LALR parsovanie

LALR je kompromis medzi SLR a kanonickým LR parsovaním. Najprv sa zostroja kanonické LR(1) položky a potom sa zlúčia stavy s rovnakým LR(0) jadrom. Lookahead množiny sa pritom zjednotia.

Napríklad stavy

$$\{[A \rightarrow \alpha \cdot \beta, a]\}$$

a

$$\{[A \rightarrow \alpha \cdot \beta, b]\}$$

sa zlúčia na

$$\{[A \rightarrow \alpha \cdot \beta, a/b]\}.$$

Počet stavov LALR parsera je rovnaký ako počet stavov SLR parsera, ale LALR parser je spravidla silnejší. Pri zlučovaní môžu vzniknúť nové reduce/reduce konflikty, ale nevznikajú nové shift/reduce konflikty.

4.21 Spracovanie chýb pri LR parsovaní

Chyba pri LR parsovaní sa deteguje pri prázdnom zázname v tabuľke ACTION, nikdy nie v tabuľke GOTO. Kanonický LR parser ohlásí chybu hneď, ako je to možné.

4.21.1 Panický mód

Pri zotavení v panickom móde:

1. vyberáme symboly zo zásobníka, kým nenájdeme stav s , ktorý má goto prechod na synchronizačný neterminál A ,
2. posúvame sa na vstupe, kým nenájdeme terminál a , ktorý môže nasledovať za A ,
3. vložíme na zásobník A a stav

$$goto[s, A],$$

4. pokračujeme v parsovaní.

Synchronizačné neterminály bývajú hlavné časti programu, napríklad výraz, príkaz alebo blok.

4.21.2 Zotavenie na úrovni frázy

Ku každému prázdnuému záznamu v parsovej tabuľke možno pridať funkciu na ošetrovanie konkrétnej chyby. Tá môže modifikovať symboly na vrchu zásobníka alebo vstupné symboly. Treba sa vyhnúť zacykleniu.

4.22 Zhrnutie

Syntaktická analýza zdola-nahor konštruje strom odvodenia od listov ku koreňu a vstup redukuje na počiatočný symbol gramatiky. Základnou technikou je shift-reduce parsovanie, ktoré používa akcie shift, reduce, accept a error. Kľúčovým pojmom je handle, teda podreťazec vhodný na redukciiu. Operátorovo-precedenčné parsovanie je jednoduchá metóda vhodná pre výrazy. Najdôležitejšou všeobecnou technikou zdola-nahor je LR parsovanie, ktoré používa tabuľky ACTION a GOTO. Metódy SLR, CLR a LALR sa líšia silou a veľkosťou tabuliek. Pri chybách sa používajú lokálne korekcie, panický mód, chybové pravidlá alebo globálne korekcie.

5 Syntaxou riadené preklady

5.1 Základná myšlienka

Syntaxou riadený preklad je spôsob, ako prepojiť syntaktickú analýzu s ďalšími fázami kompilácie. Celý preklad je riadený procesom parsovania. K pravidlám gramatiky sú priradené sémantické pravidlá alebo akcie, ktoré sa vykonávajú pri rozpoznaní syntaktických konštrukcií.

Tieto akcie môžu slúžiť napríklad na:

- generovanie medzikódu alebo cieľového kódu,
- ukladanie informácií do tabuľky symbolov,
- kontrolu typov,
- generovanie chybových hlásení,
- konštrukciu syntaktického stromu.

Používajú sa dve základné notácie:

1. syntaxou riadená definícia,
2. prekladová schéma.

5.2 Syntaxou riadená definícia

Syntaxou riadená definícia, skrátene SRD, je abstraktná špecifikácia syntaxou riadeného prekladu. Vznikne rozšírením bezkontextovej gramatiky o atribúty a sémantické pravidlá.

Atribúty Atribúty sú priradené symbolom gramatiky. Ak má symbol X atribút x , zapisujeme ho

$$X.x.$$

Atribút nesie dodatočnú informáciu o symbole, napríklad hodnotu, typ, smerník do tabuľky symbolov alebo miesto v pamäti.

Sémantické pravidlá Sémantické pravidlá sú priradené pravidlám gramatiky a určujú výpočet atribútov. Majú napríklad tvar

$$B.b := f(C_1.c_1, \dots, C_k.c_k).$$

To znamená, že atribút $B.b$ závisí od atribútov $C_1.c_1, \dots, C_k.c_k$.

Ak syntaxou riadená definícia neobsahuje funkcie s bočným efektom, nazýva sa atribútová gramatika.

5.3 Syntetizované a zdedené atribúty

5.3.1 Syntetizované atribúty

Syntetizovaný atribút uzla sa počíta z atribútov jeho detí. Informácia teda tečie smerom zdola nahor.

Príklad:

$$\begin{aligned} E &\rightarrow E_1 + T \\ E.val &:= E_1.val + T.val. \end{aligned}$$

Atribút $E.val$ je syntetizovaný, pretože sa počíta z atribútov detí $E_1.val$ a $T.val$.

5.3.2 Zdedené atribúty

Zdedený atribút uzla sa počíta z atribútov rodiča alebo súrodencov. Používa sa najmä na prenos kontextu.

Príklad deklarácie:

$$\begin{aligned} D &\rightarrow TL; \\ L.in &:= T.type. \end{aligned}$$

Tu sa typ vypočítaný v $T.type$ prenáša ako zdedený atribút $L.in$ do zoznamu identifikátorov.

5.4 S-atribútová definícia

S-atribútová definícia je syntaxou riadená definícia, ktorá obsahuje iba syntetizované atribúty.

Takáto definícia sa dá vyhodnotiť prechodom stromu odvodenia zdola nahor.

Príklad:

$$\begin{aligned}
L &\rightarrow E \ n && \text{print}(E.val), \\
E &\rightarrow E_1 + T && E.val := E_1.val + T.val, \\
E &\rightarrow T && E.val := T.val, \\
T &\rightarrow T_1 * F && T.val := T_1.val \times F.val, \\
T &\rightarrow F && T.val := F.val, \\
F &\rightarrow (E) && F.val := E.val, \\
F &\rightarrow \text{digit} && F.val := \text{digit.lexval}.
\end{aligned}$$

5.5 L-atribútová definícia

Syntaxou riadená definícia je L-atribútová, ak pre každé pravidlo

$$A \rightarrow X_1 X_2 \dots X_n$$

platí, že každý zdedený atribút symbolu X_j môže závisieť iba od:

1. atribútov symbolov $X_1, \dots, X_{j-1}$, ktoré sú v pravidle naľavo od X_j ,
2. zdedených atribútov symbolu A .

Každá S-atribútová definícia je aj L-atribútová:

$$S\text{-atribútové definície} \subset L\text{-atribútové definície}.$$

5.6 Vyhodnocovanie atribútov

Atribúty možno vyhodnocovať:

1. počas parsovania,
2. prechádzaním explicitne zostrojeného stromu odvodenia alebo syntaktického stromu.

Pri všeobecnom prístupe najprv zostrojíme strom odvodenia, potom graf závislostí atribútov a z neho určíme poradie vyhodnocovania atribútov.

5.7 Graf závislosti atribútov

Graf závislosti atribútov je orientovaný graf

$$D = (V, E),$$

kde:

- V je množina vrcholov reprezentujúcich atribúty konkrétnych uzlov stromu odvodenia,
- E je množina hrán reprezentujúcich závislosti medzi atribútmi.

Ak atribút $Y.y$ závisí od atribútu $X.x$, potom do grafu pridáme hranu

$$X.x \rightarrow Y.y.$$

Pre sémantické pravidlo

$$b := f(c_1, \dots, c_k)$$

pridáme hrany

$$c_1 \rightarrow b, \quad \dots, \quad c_k \rightarrow b.$$

5.8 Topologické triedenie a test na cirkularitu

Ak graf závislostí nemá cyklus, možno ho topologicky usporiadať. Topologické usporiadanie je poradie vrcholov

$$m_1, \dots, m_k$$

také, že ak existuje hrana

$$m_i \rightarrow m_j,$$

potom m_i je v poradí pred m_j .

Toto poradie určuje platné poradie vyhodnocovania atribútov.

Test na cirkularitu pre konkrétny strom:

1. zostroj graf závislostí atribútov,
2. pokús sa vykonať topologické triedenie,
3. ak topologické triedenie existuje, atribúty možno vyhodnotiť,
4. ak topologické triedenie neexistuje, graf obsahuje cyklus.

Ak graf obsahuje cyklus, atribúty nie je možné korektne vyhodnotiť.

5.9 Syntaktický strom

Syntaxou riadený preklad môže byť založený na strome odvodenia alebo na syntaktickom strome. Syntaktický strom je zjednodušený strom odvodenia, ktorý lepšie reprezentuje prirodzenú štruktúru programu.

Pre výrazy možno používať funkcie:

$$mknode(op, left, right),$$

$$mkleaf(id, entry),$$

$$mkleaf(num, val).$$

Príklad:

$$E \rightarrow E_1 + T$$

$$E.nptr := mknode(' + ', E_1.nptr, T.nptr).$$

$$T \rightarrow id$$

$$T.nptr := mkleaf(id, id.entry).$$

$$T \rightarrow num$$

$$T.nptr := mkleaf(num, num.val).$$

Atribút $X.nptr$ je syntetizovaný atribút obsahujúci smerník na príslušný uzol syntaktického stromu.

5.10 Prekladová schéma

Prekladová schéma je implementačnejšia forma syntaxou riadenej definície. Okrem sémantických pravidiel špecifikuje aj poradie vykonávania akcií. Sémantické akcie sa zapisujú do zátvoriek

$\{\dots\}$

a vkladajú sa priamo do pravých strán pravidiel.

Príklad prekladu infixového zápisu do postfixového:

$$E \rightarrow TR,$$

$$R \rightarrow addop\ T\ \{print(addop.lexeme)\}\ R_1\ |\ \varepsilon,$$

$$T \rightarrow num\ \{print(num.val)\}.$$

5.11 Obmedzenia pre prekladové schémy

Akcia nesmie odkazovať na atribút, ktorý ešte nebol vypočítaný.

Pre S-atribútové definície sa akcie typicky umiestňujú na koniec pravidiel.

Pre L-atribútové definície platí:

1. zdedený atribút symbolu na pravej strane pravidla musí byť vypočítaný v akcii naľavo od tohto symbolu,
2. akcia nesmie odkazovať na syntetizovaný atribút symbolu napravo,
3. syntetizovaný atribút neterminálu na ľavej strane pravidla sa zvyčajne počíta až na konci pravej strany pravidla.

5.12 Implementácia S-atribútových definícií pri LR parsovaní

S-atribútové definície sa prirodzene implementujú pri LR parsovaní. Hodnoty atribútov sú uložené v špeciálnych poliach zásobníka parsera.

Typicky máme polia:

state

a

val.

Pole *state* obsahuje stavy LR parsera a pole *val* obsahuje hodnoty atribútov asociované so symbolmi na zásobníku.

Pri redukcii podľa pravidla

$$A \rightarrow XYZ$$

so sémantickým pravidlom

$$A.a := f(X.x, Y.y, Z.z)$$

sa hodnota *A.a* vypočíta z hodnôt atribútov uložených na zásobníku:

$$Z.z = val[top],$$

$$Y.y = val[top - 1],$$

$$X.x = val[top - 2].$$

Potom sa vykoná redukcia a hodnota *A.a* sa uloží na novú pozíciu zásobníka.

5.13 Implementácia L-atribútových definícií pri prediktívnom parsovaní

L-atribútové definície sú vhodné pre prediktívne parsovanie zhora nadol. Pre každý neterminál *A* vytvoríme funkciu:

- s formálnymi parametrami pre zdedené atribúty *A*,
- s návratovou hodnotou alebo hodnotami pre syntetizované atribúty *A*.

Pri výskyte neterminálu *B* sa vygeneruje volanie:

$$c := B(b_1, b_2, \dots, b_k),$$

kde $b_1, \dots, b_k$ sú hodnoty zdedených atribútov neterminálu *B* a *c* je hodnota syntetizovaného atribútu.

5.14 Odstránenie ľavej rekurzie z prekladovej schémy

Majme prekladovú schému:

$$A \rightarrow A_1 Y \{A.a := g(A_1.a, Y.y)\},$$

$$A \rightarrow X \{A.a := f(X.x)\}.$$

Po odstránení ľavej rekurzie dostaneme gramatiku:

$$A \rightarrow XR,$$

$$R \rightarrow YR \mid \varepsilon.$$

Zodpovedajúca prekladová schéma je:

$$\begin{aligned} A &\rightarrow X \{R.i := f(X.x)\} R \{A.a := R.s\}, \\ R &\rightarrow Y \{R_1.i := g(R.i, Y.y)\} R_1 \{R.s := R_1.s\}, \\ R &\rightarrow \varepsilon \{R.s := R.i\}. \end{aligned}$$

Nový neterminál R má dva atribúty:

- zdedený atribút $R.i$, ktorý akumuluje hodnotu,
- syntetizovaný atribút $R.s$, ktorý prenáša výsledok späť.

5.15 Porovnanie metód zhora-nadol a zdola-nahor

5.15.1 Metódy zhora-nadol

Metódy zhora-nadol sú vhodné najmä pre L-atribútové definície. Zdedené atribúty možno implementovať ako parametre procedúr a syntetizované atribúty ako návratové hodnoty.

Výhody:

- jednoduchá ručná implementácia,
- prirodzené prenášanie zdedených atribútov,
- vhodné pre LL gramatiky.

Nevýhody:

- nemožno priamo použiť ľavorekuzívne gramatiky,
- často treba upraviť gramatiku aj prekladovú schému,
- LL gramatiky sú obmedzenejšie než LR gramatiky.

5.15.2 Metódy zdola-nahor

Metódy zdola-nahor sú vhodné najmä pre S-atribútové definície. Pri redukcii sú už atribúty pravej strany pravidla dostupné na zásobníku, takže možno vypočítať atribút ľavej strany.

Výhody:

- prirodzená implementácia S-atribútových definícií,
- vhodné pre LR parsovanie,
- pokrývajú väčšiu triedu gramatík než metódy zhora-nadol.

Nevýhody:

- zložitejšia implementácia,
- zdedené atribúty sa implementujú ťažšie,
- často sa používajú generátory parserov.

5.16 Zhrnutie

Syntaxou riadený preklad prepája syntaktickú analýzu s ďalšími fázami kompilácie. Syntaxou riadená definícia rozširuje bezkontextovú gramatiku o atribúty a sémantické pravidlá. Syntetizované atribúty sa počítajú z atribútov detí, zdedené atribúty z rodiča a súrodencov. S-atribútové definície obsahujú iba syntetizované atribúty a prirodzene sa vyhodnocujú zdola-nahor. L-atribútové definície povolujú aj zdedené atribúty, ale iba s obmedzenými závislosťami zľava doprava. Poradie vyhodnocovania atribútov možno určiť pomocou grafu závislostí a topologického triedenia. Prekladové schémy dopĺňajú SRD o poradie vykonávania akcií. Pri LR parsovaní sa dobre implementujú S-atribútové definície, pri prediktívnom parsovaní zhora-nadol L-atribútové definície.

6 Podpora v čase behu

6.1 Run-time environment

Run-time environment je prostredie, v ktorom sa vykonáva cieľový program vytvorený kompilátorom. Kompilátor spolu s linkerom a runtime systémom zabezpečuje, aby program dodržiaval sémantiku zdrojového jazyka.

Podpora v čase behu zahŕňa najmä:

- správu pamäti,
- volanie funkcií a procedúr,
- adresáciu dátových štruktúr,
- štandardné zabudované funkcie,
- podporu pre tabuľku symbolov.

6.2 Organizácia pamäte

Typické rozloženie pamäte programu je:

code static data heap free memory stack.

Code segment Obsahuje strojové inštrukcie programu. Kód je zvyčajne známy už počas kompilácie.

Static data Obsahuje globálne a statické premenné. Ich životnosť je zvyčajne celá doba behu programu.

Stack Zásobník obsahuje aktivačné záznamy funkcií a procedúr. Používa sa na lokálne premenné, parametre, návratové adresy, uložené registre a dočasné hodnoty.

Heap Halda sa používa na dynamicky alokované objekty, ktorých životnosť nemusí byť viazaná na jedno volanie funkcie.

6.3 Statická alokácia

Pri statickej alokácii je všetka pamäť pridelená už počas kompilácie alebo linkovania. Používa sa najmä pre kód, globálne premenné a statické dáta.

Adresa premennej môže byť určená ako

$$addr(x) = \text{začiatok oblasti} + offset(x).$$

Výhody:

- jednoduchá implementácia,
- jednoduché adresovanie.

Nevýhody:

- nepodporuje dobre rekurziu,
- každá procedúra má iba jednu inštanciu lokálnych dát.

6.4 Zásobníková alokácia

Pri zásobníkovej alokácii sa pri každom volaní funkcie vytvorí nový aktivačný záznam na zásobníku. Pri návrate z funkcie sa tento záznam odstráni.

Zásobníková alokácia umožňuje:

- rekurziu,
- viac súčasných aktivácií tej istej funkcie,
- jednoduché uvoľňovanie lokálnych premenných.

6.5 Dynamická alokácia na halde

Heap sa používa pre objekty, ktorých životnosť nie je viazaná na konkrétne volanie funkcie. Dynamická alokácia môže byť explicitná alebo implicitná.

Problémy správy heapu:

- evidencia voľných oblastí,
- fragmentácia,
- garbage, teda nedostupná alokovaná pamäť,
- dangling reference, teda ukazovateľ na už uvoľnenú pamäť.

6.6 Garbage collection

Garbage collection je automatické vyhľadávanie a uvoľňovanie nepoužitej pamäte.

6.6.1 Reference counting

Pri počítaní referencií má každý objekt počítadlo referencií. Ak počet referencií klesne na nulu, objekt je nedostupný a môže byť uvoľnený.

Výhody:

- jednoduchý princíp,
- pamäť sa často uvoľní hneď po strate poslednej referencie.

Nevýhody:

- problém s cyklickými referenciami,
- réžia pri každej zmene referencie.

6.6.2 Mark and sweep

Mark-sweep je sledovací garbage collector. Prehľadáva sa graf objektov. Postup:

1. začne sa od koreňov, napríklad registrov, zásobníka a globálnych premenných,
2. prechádza sa graf objektov,
3. každý dosiahnuteľný objekt sa označí,
4. neoznačené objekty sa uvoľnia,
5. voliteľne sa vykoná kompakcia pamäte.

6.6.3 Generational garbage collection

Generačný garbage collector delí heap na viac generácií. Mladé generácie sa prechádzajú často, staré generácie menej často. Je založený na predpoklade, že väčšina objektov má krátku životnosť.

6.7 Volanie funkcií a procedúr

Volanie funkcie musí zabezpečiť:

- odovzdanie riadenia volanej funkcii,
- odovzdanie parametrov,
- uloženie návratovej adresy,
- vytvorenie aktivačného záznamu,
- uloženie a obnovenie stavu stroja,
- návrat hodnoty,
- obnovenie vykonávania volajúcej funkcie.

6.8 Aktivačný záznam

Aktivačný záznam, alebo stack frame, obsahuje informácie potrebné pre jednu aktiváciu funkcie alebo procedúry.

Typicky obsahuje:

- parametre,
- návratovú hodnotu,
- návratovú adresu,
- uložený frame pointer a stack pointer,
- dynamický link,
- statický link,
- lokálne premenné,
- dočasné hodnoty,
- uložené registre.

6.9 Calling sequence a return sequence

Calling sequence Calling sequence je kód, ktorý pripraví volanie funkcie:

1. vytvorí aktivačný záznam volanej funkcie,
2. vyhodnotí skutočné parametre,
3. uloží parametre do aktivačného záznamu alebo registrov,
4. uloží návratovú adresu,
5. nastaví dynamický a statický link,
6. nastaví ukazovatele zásobníka,
7. odovzdá riadenie volanej funkcii.

Return sequence Return sequence obnoví stav volajúcej funkcie:

1. uloží návratovú hodnotu,
2. obnoví registre,
3. obnoví stack pointer a frame pointer,
4. obnoví program counter na návratovú adresu,
5. odstráni aktivačný záznam.

6.10 Odovzdávanie parametrov

6.10.1 Call-by-value

Pri call-by-value sa hodnota skutočného parametra skopíruje do formálneho parametra. Zmeny formálneho parametra neovplyvnia premennú vo volajúcej funkcii.

6.10.2 Call-by-reference

Pri call-by-reference sa odovzdáva adresa premennej. Volaná funkcia pracuje s pôvodnou premennou, takže zmeny sa prejavajú aj po návrate.

6.10.3 Call-by-value-result

Call-by-value-result, alebo copy-restore, najprv skopíruje hodnotu do formálneho parametra a pri návrate ju skopíruje späť do skutočného parametra.

6.10.4 Call-by-name

Pri call-by-name sa skutočný parameter správa ako textovo dosadený výraz. Vyhodnocuje sa až vtedy, keď sa použije.

6.11 Lokálne a nelokálne premenné

Premenná je lokálna, ak je deklarovaná v aktuálnej procedúre alebo bloku. Premenná je nelokálna, ak nie je deklarovaná v aktuálnom bloku, ale jej deklarácia sa nachádza v niektorom obklopujúcom scope. Globálna premenná je deklarovaná v najvonkajšom scope.

6.12 Dynamický a statický link

6.12.1 Dynamický link

Dynamický link, alebo control link, ukazuje na aktivačný záznam volajúcej procedúry:

dynamic link = aktivačný záznam caller-a.

Používa sa pri návrate z funkcie a obnovení predchádzajúceho aktivačného záznamu.

6.12.2 Statický link

Statický link, alebo access link, ukazuje na aktivačný záznam lexikálne obklopujúcej procedúry. Používa sa na prístup k nelokálnym premenným.

Adresa nelokálnej premennej môže byť reprezentovaná ako

(počet statických linkov, *offset*).

Lokálna premenná má tvar

$$(0, offset),$$

premenná z bezprostredne obklopujúceho scope má tvar

$$(1, offset).$$

6.13 Display

Display je alternatíva k statickým linkom. Je to pole ukazovateľov

$$d[1], d[2], \dots$$

kde $d[i]$ ukazuje na aktuálny aktivačný záznam procedúry na úrovni vnorenia i .

Ak procedúra potrebuje premennú zo scope s hĺbkou i , jej adresa sa získa:

$$addr(x) = d[i] + offset(x).$$

Výhody:

- rýchly prístup k nelokálnym premenným,
- netreba prechádzať reťaz statických linkov.

Nevýhody:

- treba udržiavať dodatočné pole,
- pri volaní a návrate treba ukladať a obnovovať staré hodnoty.

6.14 Scope

Scope je časť programu, v ktorej sa dané meno vzťahuje na konkrétnu deklaráciu.

6.14.1 Statický, lexikálny scope

Pri statickom scope sa väzba mena na deklaráciu určuje podľa textovej štruktúry programu. Používa sa pravidlo najbližšieho vnorenia: ak meno nie je deklarované v aktuálnom bloku, hľadá sa v najbližšom obklopujúcom bloku.

6.14.2 Dynamický scope

Pri dynamickom scope sa väzba mena určuje podľa aktuálneho volacieho reťazca počas behu programu. Meno sa hľadá najprv v aktuálnom aktivačnom zázname, potom v aktivačnom zázname volajúcej funkcie atď.

6.15 Statický vs. dynamický scope

Majme program:

```
var x := 1;
procedure A(){print(x);}
procedure B(){var x := 2; A();}
```

Pri volaní

B()

statický scope použije globálne *x*, pretože procedúra *A* je textovo definovaná v globálnom scope. Výstup je:

1.

Dynamický scope použije *x* z procedúry *B*, pretože *B* je aktuálny volajúci. Výstup je:

2.

6.16 Zhrnutie

Run-time environment zabezpečuje správu pamäti, volanie funkcií, adresáciu dát a podporu pravidiel jazyka počas behu programu. Pamäť programu sa typicky delí na code, static data, heap a stack. Statická alokácia je jednoduchá, ale nepodporuje dobre rekurziu. Zásobníková alokácia používa aktivačné záznamy a podporuje rekurziu. Heap slúži pre dynamicky alokované objekty a môže byť spravovaný manuálne alebo pomocou garbage collection. Aktivačný záznam obsahuje parametre, návratovú adresu, linky, lokálne dáta a dočasné hodnoty. Dynamický link ukazuje na caller-a, statický link na lexikálne obklopujúcu aktiváciu. Display zrýchľuje prístup k nelokálnym premenným. Statický scope sa určuje podľa textového vnorenia programu, dynamický scope podľa aktuálneho volacieho reťazca.

7 Generovanie medzijazyka

7.1 Úloha medzijazyka

Medzijazyk je vnútorná reprezentácia programu medzi zdrojovým jazykom a cieľovým kódom. Používa sa preto, že:

- je jednoduchšie generovať medzikód než priamo strojový kód,
- nad medzikódom sa ľahšie vykonávajú optimalizácie,
- oddeľuje front-end kompilátora od back-endu,
- je menej závislý od konkrétnej cieľovej architektúry.

7.2 Formy medzijazyka

7.2.1 Poľská sufixová forma

Poľská sufixová forma, teda Reverse Polish Notation, zapisuje operátor za jeho operandy. Napríklad výraz

$$a := b * c + b * c$$

môže byť zapísaný ako

$$a \ b \ c \ * \ b \ c \ * \ + \ := \ .$$

Výhodou je jednoduché vyhodnocovanie zásobníkovým automatom. Nevýhodou sú malé možnosti optimalizácie.

7.2.2 Syntaktické stromy a DAG

Syntaktický strom reprezentuje štruktúru výrazu alebo programu. Vnútorne uzly zodpovedajú operáciám a listy identifikátorom alebo konštantám.

DAG, teda orientovaný acyklický graf, umožňuje zdieľať spoločné podvýrazy. Napríklad vo výraze

$$b * c + b * c$$

sa podvýraz $b * c$ uloží iba raz.

7.2.3 Trojadresový kód

Trojadresový kód je lineárna reprezentácia programu. Každá inštrukcia má najviac jeden operátor na pravej strane:

$$x := y \ op \ z.$$

Adresy môžu byť:

- mená premenných,
- konštanty,
- dočasné premenné.

Typické inštrukcie:

$x := y \ op \ z$	binárna operácia,
$x := op \ y$	unárna operácia,
$x := y$	kopírovanie,
$goto \ L$	nepodmienený skok,
$if \ x \ rel \ op \ y \ goto \ L$	podmienený skok,
$param \ x$	parameter volania,
$call \ p \ n$	volanie procedúry alebo funkcie,
$return \ y$	návrat hodnoty,
$x := y[i]$	čítanie z poľa,
$x[i] := y$	zápis do poľa.

7.3 Štvorice, trojice a nepriame trojice

7.3.1 Štvorice

Štvorica má tvar

$$(op, arg_1, arg_2, result).$$

Napríklad inštrukcia

$$t_1 := b * c$$

sa reprezentuje ako

$$(*, b, c, t_1).$$

Štvorice sú prehľadné a vhodné na optimalizácie, pretože výsledok inštrukcie má explicitné meno.

7.3.2 Trojice

Trojica má tvar

$$(op, arg_1, arg_2).$$

Výsledok inštrukcie sa neukladá do explicitného poľa, ale odkazuje sa číslom trojice.

Príklad:

$$\begin{array}{c|cc} 0 & id & b \\ 1 & id & c \\ 2 & * & 0 \quad 1 \end{array}$$

Výhodou je menšia pamäťová náročnosť. Nevýhodou je, že preusporiadanie inštrukcií sťažuje optimalizácie.

7.3.3 Nepriame trojice

Nepriame trojice používajú tabuľku trojíc a samostatný zoznam ukazovateľov na trojice. Pri optimalizácii možno preusporiadať ukazovatele namiesto samotných trojíc.

7.4 Generovanie medzijazyka syntaxou riadeným prekladom

Na generovanie trojadresového kódu sa používa syntaxou riadený preklad.

Príkaz S má atribút

$$S.code,$$

ktorý obsahuje vygenerovaný kód.

Výraz E má atribúty

$$E.code$$

a

$$E.place.$$

Atribút $E.code$ obsahuje kód potrebný na výpočet výrazu a atribút $E.place$ označuje miesto, kde je uložená hodnota výrazu.

Používajú sa pomocné funkcie:

- *newtemp* vytvorí novú dočasnú premennú,
- *newlabel* vytvorí nový label,
- *gen* vygeneruje inštrukciu,
- $||$ označuje zretazenie fragmentov kódu.

7.5 Aritmetické výrazy

Pre pravidlo

$$E \rightarrow E_1 + E_2$$

platí:

$$E.place := newtemp,$$

$$E.code := E_1.code || E_2.code || gen(E.place := E_1.place + E_2.place).$$

Pre pravidlo

$$E \rightarrow E_1 * E_2$$

platí:

$$E.place := newtemp,$$

$$E.code := E_1.code || E_2.code || gen(E.place := E_1.place * E_2.place).$$

Pre unárne mínus:

$$E \rightarrow -E_1$$

$$E.place := newtemp,$$

$$E.code := E_1.code || gen(E.place := uminus E_1.place).$$

Pre identifikátor:

$$E \rightarrow id$$

$$E.place := id.place,$$

$$E.code := \varepsilon.$$

7.6 Príkaz priradenia

Pre pravidlo

$$S \rightarrow id := E$$

generujeme:

$$S.code := E.code || gen(id.place := E.place).$$

Príklad:

$$a := b * c + b * (-c)$$

môže dať kód:

$$\begin{aligned} t_1 &:= b * c, \\ t_2 &:= uminus\ c, \\ t_3 &:= b * t_2, \\ t_4 &:= t_1 + t_3, \\ a &:= t_4. \end{aligned}$$

7.7 Polia

Pre jednorozmerné pole $A[low : high]$, kde w je veľkosť prvku, je adresa prvku $A[i]$:

$$base + (i - low)w.$$

Ekvivalentne:

$$base - low \cdot w + i \cdot w = A[0] + i \cdot w.$$

Pre dvojrozmerné pole v row-major poradí:

$$A[i_1, i_2]$$

je adresa:

$$A[0, 0] + (i_1 n_2 + i_2)w,$$

kde n_2 je počet prvkov druhej dimenzie.

Na prístup k poliam sa používajú inštrukcie:

$$x := y[i],$$

$$x[i] := y.$$

7.8 Logické výrazy

7.8.1 Klasický spôsob

Pri klasickom spôsobe sa hodnota logického výrazu uloží ako 0 alebo 1.

Pre

$$E \rightarrow E_1 \text{ and } E_2$$

môžeme generovať:

$$E.place := newtemp,$$

$$gen(E.place := E_1.place \text{ and } E_2.place).$$

Pre relačný výraz:

$$E \rightarrow aE_1 \text{ relop } aE_2$$

môžeme generovať:

$$\begin{aligned} & \text{if } aE_1.place \text{ relop } aE_2.place \text{ goto } +3, \\ & E.place := 0, \\ & \text{goto } +2, \\ & E.place := 1. \end{aligned}$$

7.8.2 Skratkové vyhodnocovanie

Pri skratkovom vyhodnocovaní sa negeneruje hodnota 0 alebo 1, ale skoky na pravdivú a nepravdivú vetvu.

Výraz má atribúty:

$E.true, \quad E.false, \quad E.code.$

Pre relačný výraz generujeme:

$if \ aE_1.place \ relop \ aE_2.place \ goto \ E.true,$
 $goto \ E.false.$

7.9 Riadiace konštrukcie

7.9.1 If-then

Pre pravidlo

$S \rightarrow if \ E \ then \ S_1$

nastavíme:

$E.true := newlabel,$
 $E.false := S.next,$
 $S_1.next := S.next.$

Kód:

$E.code,$
 $E.true \ ;,$
 $S_1.code.$

7.9.2 If-then-else

Pre pravidlo

$S \rightarrow if \ E \ then \ S_1 \ else \ S_2$

nastavíme:

$E.true := newlabel, \quad E.false := newlabel,$
 $S_1.next := S.next, \quad S_2.next := S.next.$

Kód:

$E.code,$
 $E.true \ ;,$
 $S_1.code,$
 $goto \ S.next,$
 $E.false \ ;,$
 $S_2.code.$

7.9.3 While

Pre pravidlo

$$S \rightarrow \text{while } E \text{ do } S_1$$

nastavíme:

$$\begin{aligned} S.begin &:= \text{newlabel}, \\ E.true &:= \text{newlabel}, \\ E.false &:= S.next, \\ S_1.next &:= S.begin. \end{aligned}$$

Kód:

$$\begin{aligned} &S.begin :, \\ &E.code, \\ &E.true :, \\ &S_1.code, \\ &\text{goto } S.begin. \end{aligned}$$

7.10 Volanie procedúr a funkcií

Pre pravidlo

$$S \rightarrow \text{call } id(L)$$

sa najprv vygenerujú parametre a potom samotné volanie.

Pre zoznam parametrov:

$$L \rightarrow L, E$$

vložíme $E.place$ do fronty parametrov.

Pre

$$L \rightarrow E$$

inicializujeme frontu a vložíme $E.place$.

Potom generujeme:

$$\begin{aligned} &\text{param } p_1, \\ &\text{param } p_2, \\ &\vdots \\ &\text{call } id.place. \end{aligned}$$

Pri funkcii s návratovou hodnotou môže byť výsledok uložený do dočasnej premennej:

$$t := \text{call } f, n.$$

7.11 Spätné plátanie

Spätné plátanie, teda backpatching, sa používa pri jednoprechodovom generovaní kódu pre dopredné skoky. Ak cieľ skoku ešte nepoznáme, vygenerujeme skok s prázdnyim cieľom a pozíciu tejto inštrukcie uložíme do zoznamu. Keď je cieľ známy, doplníme ho.

Používajú sa funkcie:

$$makelist(i),$$

ktorá vytvorí zoznam obsahujúci inštrukciu i ,

$$merge(p_1, p_2),$$

ktorá spojí dva zoznamy,

$$backpatch(p, i),$$

ktorá doplní cieľ i do všetkých inštrukcií v zozname p ,

$$nextquad,$$

ktorý označuje adresu nasledujúcej generovanej inštrukcie.

7.11.1 Backpatching pre logické výrazy

Logický výraz má atribúty:

$$E.truelist, \quad E.falselist.$$

Pre relačný výraz

$$E \rightarrow aE_1 \text{ relop } aE_2$$

generujeme:

$$\begin{aligned} E.truelist &:= makelist(nextquad), \\ E.falselist &:= makelist(nextquad + 1), \\ gen(if \ aE_1.place \ relop \ aE_2.place \ goto \ -), \\ gen(goto \ -). \end{aligned}$$

Pomocný neterminál:

$$M \rightarrow \varepsilon$$

má sémantické pravidlo:

$$M.quad := nextquad.$$

Pre disjunkciu:

$$E \rightarrow E_1 \text{ or } M \ E_2$$

platí:

$$\begin{aligned} backpatch(E_1.falselist, M.quad), \\ E.truelist := merge(E_1.truelist, E_2.truelist), \end{aligned}$$

$$E.falselist := E_2.falselist.$$

Pre konjunkciu:

$$E \rightarrow E_1 \text{ and } M \ E_2$$

platí:

$$\begin{aligned} & \text{backpatch}(E_1.truelist, M.quad), \\ & E.truelist := E_2.truelist, \\ & E.falselist := \text{merge}(E_1.falselist, E_2.falselist). \end{aligned}$$

7.11.2 Backpatching pre príkazy

Pre

$$S \rightarrow \text{if } E \text{ then } M \ S_1$$

platí:

$$\begin{aligned} & \text{backpatch}(E.truelist, M.quad), \\ & S.nextlist := \text{merge}(E.falselist, S_1.nextlist). \end{aligned}$$

Pre

$$S \rightarrow \text{while } M_1 \ E \ \text{do } M_2 \ S_1$$

platí:

$$\begin{aligned} & \text{backpatch}(S_1.nextlist, M_1.quad), \\ & \text{backpatch}(E.truelist, M_2.quad), \\ & S.nextlist := E.falselist, \\ & \text{gen}(\text{goto } M_1.quad). \end{aligned}$$

7.12 Zhrnutie

Generovanie medzijazyka vytvára vnútornú reprezentáciu programu vhodnú na optimalizácie a následné generovanie cieľového kódu. Medzikód môže mať formu postfixového zápisu, syntaktického stromu, DAGu alebo trojadresového kódu. Trojadresový kód možno reprezentovať štvoricami, trojicami alebo nepriamymi trojicami. Pri syntaxou riadenom preklade majú výrazy atribúty *code* a *place*, príkazy atribút *code*. Riadiace konštrukcie sa generujú pomocou labelov a skokov. Backpatching rieši dopredné skoky pri jednoprechodovom generovaní tým, že cieľ skokov sa dopĺňajú dodatočne.

8 Generovanie kódu

8.1 Úloha generátora kódu

Generátor kódu prekladá medzikód do cieľového jazyka, typicky assembleru alebo strojového kódu. Jeho úlohou je:

- vybrať cieľové inštrukcie,
- prideliť premenné a dočasné hodnoty do registrov,
- zabezpečiť prístup do pamäte,
- generovať kód pre základné bloky,
- zohľadniť cenu inštrukcií a vlastnosti cieľovej architektúry.

Ak nemáme vysoké nároky na kvalitu kódu, môžeme každú inštrukciu medzikódu nahradiť pevnou šablónou cieľových inštrukcií. Kvalitnejší generátor sa však snaží znížiť počet inštrukcií, prístupov do pamäte a zbytočných presunov medzi registrami.

8.2 Cieľový počítač

8.2.1 RISC

RISC, teda Reduced Instruction Set Computer, má typicky:

- jednoduchšie inštrukcie,
- veľa registrov,
- jednoduché adresné módy,
- pevnejšiu dĺžku inštrukcií,
- load/store model,
- jednoduchšie prúdové spracovanie.

Výpočtové operácie pracujú najmä s registrami. Prístup do pamäte je zabezpečený samostatnými inštrukciami typu LD a ST.

8.2.2 CISC

CISC, teda Complex Instruction Set Computer, má typicky:

- zložitejšie inštrukcie,
- menej registrov,
- viac adresných módov,

- variabilnú dĺžku inštrukcií,
- možnosť pracovať priamo s operandmi v pamäti.

CISC môže mať menší počet inštrukcií vo výslednom programe, ale jednotlivé inštrukcie môžu byť drahšie a zložitejšie na pipelining.

8.3 Čo môže generátor kódu generovať

Generátor kódu môže generovať:

- absolútny strojový kód,
- binárny relokatívny kód,
- assembler.

Relokatívny kód umožňuje, aby bol program umiestnený kdekoľvek v pamäti. Umiestnenie a opravu adries zabezpečuje linker alebo loader.

8.4 Dĺžka a cena inštrukcií

Cena inštrukcie závisí najmä od počtu pamäťových cyklov. Treba zohľadniť:

- načítanie inštrukcie,
- dekódovanie inštrukcie,
- výpočet adresy,
- načítanie operandov,
- vykonanie operácie,
- zápis výsledku.

Dôležitú úlohu majú aj šírka dátovej a inštrukčnej zbernice, počet registrov, počet adresných módov a veľkosť priamo adresovateľnej pamäte.

8.5 Adresné módy

Typické adresné módy:

mód	forma	adresa
absolute	M	M
register	R	R
indexed	$c(R)$	$c + contents(R)$
indirect register	$*R$	$contents(R)$
indirect indexed	$*c(R)$	$contents(c + contents(R))$
literal	$\#c$	c
stack	SP	SP
indexed stack	$c(SP)$	$c + contents(SP)$

Adresné módy sú dôležité pri generovaní kódu pre polia, lokálne premenné a prístup cez smerníky.

8.6 Cache a pipeline

Moderné procesory používajú cache a pipeline. Kompilátor by mal zohľadniť:

- dátovú a inštrukčnú lokalitu,
- poradie prístupov do pamäte,
- závislosti medzi inštrukciami,
- vetvenie a skoky.

Pipeline môže mať fázy:

$IF, ID, MA, EX, WB,$

kde IF je načítanie inštrukcie, ID dekódovanie, MA prístup do pamäte, EX vykonanie a WB zápis výsledku.

Konflikty v pipeline:

- štrukturálne konflikty,
- dátové konflikty,
- riadiace konflikty.

Dátové a štrukturálne konflikty možno riešiť pozdržaním. Riadiace konflikty pri skokoch môžu vyžadovať zrušenie časti výpočtu.

8.7 Administrácia pamäte

Pre každú procedúru sú dôležité tri oblasti:

- oblasť kódu,
- oblasť statických dát,
- oblasť dát na zásobníku.

Heap je spoločná oblasť pre dynamicky alokované dáta.

Statická pamäť Obsahuje globálne a statické premenné. Adresy sú známe pri kompilácii alebo linkovaní.

Zásobník Obsahuje aktivačné záznamy procedúr. Používa sa na parametre, lokálne premenné, návratové adresy a uložené registre.

Heap Používa sa na dynamicky alokované objekty. Generátor kódu môže generovať volania runtime rutín pre alokáciu a dealokáciu.

8.8 Rozdelenie programu na základné bloky

Základný blok je postupnosť za sebou idúcich príkazov, ktorá sa po začatí vykoná celá.

Hlavičky blokov, teda leadery, určíme takto:

1. prvý príkaz programu je leader,
2. každý cieľ skoku je leader,
3. každý príkaz bezprostredne nasledujúci za skokom je leader,
4. iné leadery neexistujú.

Blok je úsek programu od jedného leadera po nasledujúci leader.

8.9 Next-use information

Definícia

$$p : x := \dots$$

má ďalšie použitie, ak existuje príkaz s , ktorý používa x , a existuje cesta z p do s , na ktorej sa hodnota x nezmení.

Premenná, ktorá má ďalšie použitie, je živá. Premenná, ktorá nemá ďalšie použitie, je mŕtva.

Výpočet v základnom bloku sa robí odzadu:

```
for  $i$  = last statement of  $B$  downto first statement of  $B$  :  
  if  $i$  má tvar  $x := op(x_1, \dots, x_k)$  then  
    pre  $i$  nastav next-use podľa tabuľky symbolov;  
    nastav  $x$  ako neživé;  
    nastav  $x_1, \dots, x_k$  ako živé a ich next-use na  $i$ .
```

8.10 DAG základného bloku

DAG základného bloku sa vytvára podľa pravidiel:

1. pre každú pôvodnú hodnotu premennej vytvoríme list,
2. pre každý príkaz vytvoríme uzol,
3. synovia uzla sú posledné definície operandov pred daným príkazom,
4. uzol označíme operáciou a zoznamom premenných, pre ktoré je poslednou definíciou,
5. uzly živé na výstupe z bloku označíme ako výstupné.

Použitie DAGu:

- eliminácia lokálnych spoločných podvýrazov,

- eliminácia mŕtveho kódu,
- preusporiadanie nezávislých príkazov,
- využitie algebraických zákonov.

Pri poliach, smerníkoch a volaniach procedúr treba byť opatrný, pretože môžu používať alebo meniť pamäť spôsobom, ktorý nie je lokálne zrejmý.

8.11 Pridelovanie registrov

Pridelovanie registrov má dve časti:

- *register allocation*: určenie, ktoré premenné budú v registroch,
- *register assignment*: výber konkrétnych registrov.

Registre sú dôležité, pretože väčšina architektúr vyžaduje, aby aspoň jeden operand výpočtovej inštrukcie bol v registri. Registre sú vhodné na medzivýsledky, dočasné premenné, premenné cyklov a runtime ukazovatele.

8.12 Register descriptor a address descriptor

Register descriptor Pre každý register eviduje, ktoré premenné alebo hodnoty sa v ňom nachádzajú.

Address descriptor Pre každú premennú eviduje miesta, kde sa nachádza jej aktuálna hodnota. Môže ísť o register, pamäť, zásobník alebo heap.

8.13 Jednoduchý algoritmus generovania kódu

Pre každú inštrukciu medzikódu v základnom bloku:

```

for  $i = leader_B$  to  $nextleader$  do
     $LR := GetReg(instruction[i]);$ 
     $CodeGen(instruction[i], LR);$ 
     $Update(register\_descriptor);$ 
     $Update(address\_descriptor).$ 

```

Procedúry *GetReg* a *CodeGen* využívajú informácie z registrových a adresových deskriptorov, aby sa negenerovali zbytočné presuny.

8.14 Stratégia GetReg

Pre inštrukciu

$$x := y \text{ op } z$$

najprv hľadáme registre pre operandy y a z :

1. ak je operand už v registri, použijeme tento register,

2. inak použijeme register s mŕtvou premennou,
3. inak použijeme prázdny register,
4. inak môžeme použiť register obsahujúci x , ak ho aj tak prepíšeme,
5. inak vyberieme register na spill.

Ak je hodnota premennej uložená aj inde, register možno prepísať. Ak je register jediné miesto s aktuálnou hodnotou, pred prepísaním treba vygenerovať inštrukciu ST.

8.15 Globálne pridelovanie registrov

Pri lokálnom pridelovaní sa na konci bloku živé premenné ukladajú do pamäte. Pri cykloch je výhodné držať často používané premenné v registroch naprieč viacerými blokmi.

Odhad zisku držania premennej x v registri počas cyklu L :

$$\sum_{B \in L} (use(x, B) + 2 \cdot live(x, B)).$$

8.16 Pridelovanie registrov farbením grafu

Register interference graph je graf, kde:

- vrcholy reprezentujú premenné alebo dočasné hodnoty,
- hrana medzi vrcholmi znamená, že hodnoty sú živé súčasne,
- susedné vrcholy nemôžu dostať rovnaký register.

Ak máme k registrov, snažíme sa graf zafarbiť k farbami. Farba predstavuje register. Ak graf nemožno zafarbiť k farbami, niektoré premenné treba spillnúť do pamäte.

Zjednodušený algoritmus:

1. zostroj graf interferencií,
2. opakovane odstraňuj vrcholy so stupňom menším než k a ukladaj ich na zásobník,
3. ak taký vrchol neexistuje, vyber kandidáta na spill,
4. vracaj vrcholy zo zásobníka a priraďuj im registre,
5. ak nemožno priradiť register, vlož spill kód a proces zopakuj.

8.17 Generovanie kódu pokrývaním ozdobeného stromu

Generovanie kódu možno chápať ako pokrývanie stromu vzormi inštrukcií, teda *tiling*.

Dané sú:

- ozdobený syntaktický strom alebo strom výrazu,
- množina kachličiek, teda inštrukcií alebo skupín inštrukcií,
- cena každej kachličky.

Cieľom je nájsť najlacnejšie pokrytie stromu.

Postup:

1. nájdí vzor v strome,
2. nahraď ho výsledkom, typicky registrom,
3. umiestni kachličku a pripočítaj jej cenu,
4. opakuj, kým celý strom nie je pokrytý.

Optimálne pokrytie stromu možno riešiť dynamickým programovaním.

8.18 Zhrnutie

Generovanie kódu prekladá medzikód do cieľového kódu. Musí zohľadniť cieľovú architektúru, cenu inštrukcií, adresné módy, registre, cache, pipeline a správu pamäte. Program sa často delí na základné bloky, nad ktorými sa vytvára DAG na lokálne optimalizácie. Kľúčovou úlohou je pridelovanie registrov, ktoré možno riešiť lokálne pomocou deskriptorov alebo globálne pomocou farbenia grafu interferencií. Výber inštrukcií možno formulovať ako pokrývanie ozdobeného stromu kachličkami s cenami.

9 Optimalizácia kódu

9.1 Základná charakteristika

Optimalizácia kódu je fáza kompilátora, ktorá transformuje medzikód alebo cieľový kód tak, aby sa zlepšila jeho efektívnosť bez zmeny významu programu.

Optimalizovať môžeme:

- rýchlosť vykonávania,
- veľkosť kódu,
- počet prístupov do pamäte,
- využitie registrov.

Podľa rozsahu rozlišujeme:

- lokálnu optimalizáciu v rámci základného bloku,
- globálnu intraprocedurálnu optimalizáciu,
- interprocedurálnu optimalizáciu.

9.2 Základný blok

Základný blok je maximálna postupnosť príkazov, ktorá sa po začatí vykoná celá. Vstup do bloku je iba cez jeho prvý príkaz a skok sa môže nachádzať najviac na konci bloku.

V grafe toku riadenia sú uzly základné bloky a hrany reprezentujú možné prechody riadenia medzi blokmi.

9.3 Optimalizácia základných blokov pomocou DAGu

DAG základného bloku je orientovaný acyklický graf reprezentujúci závislosti medzi výpočtami v bloku.

Pravidlá vytvorenia DAGu:

1. Pre každú pôvodnú hodnotu premennej vytvoríme list.
2. Pre každý príkaz bloku vytvoríme uzol.
3. Synovia uzla sú posledné definície operandov pred daným príkazom.
4. Uzol označíme operáciou a zoznamom premenných, pre ktoré je poslednou definíciou v bloku.
5. Uzly, ktorých premenné sú živé na výstupe z bloku, označíme ako výstupné.

DAG umožňuje:

- elimináciu lokálnych spoločných podvýrazov,
- elimináciu mŕtveho kódu,
- preusporiadanie nezávislých príkazov,
- využitie algebraických zákonov.

9.4 Eliminácia spoločných podvýrazov

Spoločný podvýraz je výraz, ktorý sa v bloku počíta viackrát a jeho operandy sa medzi jednotlivými výpočtami nezmenili.

Príklad:

$$\begin{aligned}a &:= b + c, \\ b &:= a - d, \\ c &:= b + c, \\ d &:= a - d.\end{aligned}$$

Ak sa výraz $a - d$ už vypočítal a hodnoty a a d sa nezmenili, druhý výpočet možno nahradiť použitím už vypočítanej hodnoty.

9.5 Eliminácia mŕtveho kódu

Mŕtvy kód je kód, ktorého výsledok sa už nikdy nepoužije, alebo kód, ktorý sa nikdy nevykoná.

V DAGu sa mŕtvy kód odstraňuje tak, že postupne odstraňujeme korene, ktoré nemajú asociované živé premenné.

Príklad:

$$\begin{aligned}a &:= b + c, \\ b &:= b - d, \\ c &:= c + d, \\ e &:= b + c.\end{aligned}$$

Ak po výstupe z bloku sú živé iba a a b , potom výpočty pre c a e môžu byť odstránené.

9.6 Opatrnosť pri poliach, smerníkoch a procedúrach

Pri poliach, smerníkoch a volaniach procedúr nemožno vždy bezpečne preusporiadať príkazy alebo znovu použiť starú hodnotu.

Príklad:

$$\begin{aligned}x &:= a[i], \\ a[j] &:= y, \\ z &:= a[i].\end{aligned}$$

Nemôžeme nahradiť posledné čítanie $a[i]$ hodnotou x , pretože môže platiť $i = j$.

Podobne:

- čítanie cez smerník môže používať neznámu premennú,
- zápis cez smerník môže meniť neznámu premennú,
- volanie procedúry môže mať vedľajšie efekty.

9.7 Algebraické identity

Algebraické identity umožňujú zjednodušiť výrazy:

$$x + 0 = x,$$

$$0 + x = x,$$

$$x - 0 = x,$$

$$1 \cdot x = x,$$

$$x \cdot 1 = x,$$

$$x/1 = x.$$

9.8 Redukcia v sile

Redukcia v sile nahrádza drahšie operácie lacnejšími.

Príklady:

$$x^2 \rightarrow x \cdot x,$$

$$2x \rightarrow x + x,$$

$$2x \rightarrow x << 1.$$

V cykloch možno často nahradiť opakované násobenie postupným pripočítavaním.

9.9 Vyhodnocovanie konštantných podvýrazov

Vyhodnocovanie konštantných podvýrazov, teda constant folding, počíta výrazy zložené iba z konštant už počas kompilácie.

Príklad:

$$\text{int } x = 2 + 3 * 4$$

sa zmení na:

$$\text{int } x = 14.$$

9.10 Peephole optimalizácia

Peephole optimalizácia sleduje malé okienko vygenerovaného kódu a nahrádza neefektívne sekvencie efektívnejšími.

Typické prípady:

- eliminácia nedostupného kódu,
- optimalizácia toku riadenia,
- odstránenie skoku na skok,
- odstránenie zbytočných LD a ST inštrukcií,
- využitie algebraických zákonov,
- využitie špeciálnych inštrukcií cieľového stroja.

Príklad:

$$\begin{array}{l} LD \ R_0, a, \\ ST \ a, R_0 \end{array}$$

je zbytočné, ak medzi týmito inštrukciami nenastala zmena.

Príklad využitia strojového idiomu:

$$a := a + 1$$

možno nahradiť:

$$INC \ a.$$

9.11 Analýza toku dát

Analýza toku dát zisťuje počas kompilácie vlastnosti programu, ktoré sa týkajú jeho správania počas behu.

Používa sa graf toku riadenia:

- uzly sú základné bloky,
- hrany sú možné prechody riadenia,
- $pred(B)$ je množina predchodcov bloku B ,
- $succ(B)$ je množina nasledovníkov bloku B .

9.12 Hodnoty toku dát a prenosové funkcie

Pre každý blok B definujeme:

$$IN[B],$$

informáciu na vstupe do bloku,

$$OUT[B],$$

informáciu na výstupe z bloku,

$$f_B,$$

prenosovú funkciu bloku.

Ak blok pozostáva z príkazov

$$s_1, s_2, \dots, s_n,$$

potom:

$$f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}.$$

9.13 Reaching definitions

Reaching definitions určujú, ktoré definície premenných môžu dosiahnuť daný bod programu.

Definícia premennej x je príkaz tvaru:

$$x := \dots$$

Definícia dosahuje bod programu, ak existuje cesta od definície do tohto bodu, na ktorej sa premenná x nepredefinuje.

Pre príkaz:

$$s : x := y + z$$

platí:

$$OUT[s] = \{s\} \cup (IN[s] - \{\text{predchádzajúce definície } x\}).$$

Pre blok:

$$OUT[B] = gen_B \cup (IN[B] - kill_B).$$

Rovnice toku dát:

$$IN[B] = \bigcup_{P \in pred(B)} OUT[P],$$

$$OUT[B] = gen_B \cup (IN[B] - kill_B).$$

Ide o doprednú may analýzu.

9.14 Živosť premenných

Premenná je živá v danom bode, ak sa jej aktuálna hodnota môže ešte v budúcnosti použiť pred tým, než bude prepísaná.

Pre príkaz:

$$s : x := y + z$$

platí:

$$IN[s] = (OUT[s] - \{x\}) \cup \{y, z\}.$$

Pre blok:

$$OUT[B] = \bigcup_{S \in succ(B)} IN[S],$$

$$IN[B] = use_B \cup (OUT[B] - def_B).$$

Ide o spätnú analýzu.

9.15 Available expressions

Available expressions určujú výrazy, ktoré boli vypočítané na každej ceste do daného bodu a ich operandy sa odvtedy nezmenili.

Ide o doprednú must analýzu. Pri spájaní ciest sa používa prienik:

$$IN[B] = \bigcap_{P \in pred(B)} OUT[P].$$

Prenosová funkcia:

$$OUT[B] = gen_B \cup (IN[B] - kill_B).$$

9.16 May a must analýzy

Pri may analýze zisťujeme, či vlastnosť môže platiť aspoň na jednej ceste. Pri spájaní ciest používame zjednotenie:

$$\bigcup.$$

Príkladom je reaching definitions.

Pri must analýze zisťujeme, či vlastnosť platí na všetkých cestách. Pri spájaní ciest používame prienik:

$$\bigcap.$$

Príkladom sú available expressions a definitely assigned variables.

9.17 Rovnice toku dát

Všeobecné rovnice pre doprednú analýzu:

$$IN[B_0] = I_0,$$

$$OUT[B] = f_B(IN[B]),$$

$$IN[B] = \bigsqcup_{P \in pred(B)} OUT[P].$$

Všeobecné rovnice pre spätnú analýzu:

$$OUT[B_F] = I_F,$$

$$IN[B] = f_B(OUT[B]),$$

$$OUT[B] = \bigsqcup_{S \in succ(B)} IN[S].$$

Operácia $\sqcup$ je spojenie podľa typu analýzy, napríklad zjednotenie alebo prienik.

9.18 Iteratívne riešenie rovníc toku dát

Rovnice toku dát riešime hľadaním pevného bodu. Pre reaching definitions:

```

OUT[ENTRY] := ∅;
for each B ≠ ENTRY do OUT[B] := ∅;
while nastane zmena do
  for each B ≠ ENTRY do
    IN[B] :=  $\bigcup_{P \in pred(B)} OUT[P]$ ;
    OUT[B] :=  $gen_B \cup (IN[B] - kill_B)$ .

```

Algoritmus skonverguje, pretože doména je konečná a prenosové funkcie sú monotónne.

9.19 Worklist algoritmus

Efektívnejšia verzia používa pracovný zoznam blokov, ktorých sa zmena môže týkať.

```
for each  $B$  do  $OUT[B] := f_B(\perp)$ ;  
 $IN[B_0] := I_0$ ;  
 $OUT[B_0] := f_{B_0}(I_0)$ ;  
 $worklist := Blocks - \{B_0\}$ ;  
while  $worklist \neq \emptyset$  do  
    vyber blok  $B$  z  $worklist$ ;  
     $IN[B] := \bigsqcup_{P \in pred(B)} OUT[P]$ ;  
     $old := OUT[B]$ ;  
     $OUT[B] := f_B(IN[B])$ ;  
    if  $OUT[B] \neq old$  then  
         $worklist := worklist \cup succ(B)$ .
```

9.20 Zhrnutie

Optimalizácia kódu zachováva význam programu, ale zlepšuje jeho efektívnosť. Lokálne optimalizácie sa robia v rámci základných blokov, často pomocou DAGu. DAG umožňuje eliminovať spoločné podvýrazy, mŕtvy kód a preusporiadať nezávislé príkazy. Ďalšie transformácie sú algebraické identity, redukcia v sile, vyhodnocovanie konštánt a peephole optimalizácia. Globálne optimalizácie potrebujú analýzu toku dát. Tá pracuje nad grafom toku riadenia a počíta množiny IN a OUT pomocou prenosových funkcií. Rovnice toku dát sa riešia iteráciou alebo efektívnejším worklist algoritmom.